
Captura de movimientos para la detección de
gestos en entornos virtuales
Motion capture for gesture detection in virtual
environments



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Alejandro Barrachina Argudo, Pablo Sánchez Martín

Director

Alejandro Romero Hernández, Ismael Sagredo Olivenza

Colaborador

Grado en Ingeniería Informática y Grado en Desarrollo
de Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

Captura de movimientos para la detección
de gestos en entornos virtuales
Motion capture for gesture detection in
virtual environments

Trabajo de Fin de Grado en Ingeniería Informática y Grado
en Desarrollo de Videojuegos

Autor

Alejandro Barrachina Argudo, Pablo Sánchez Martín

Director

Alejandro Romero Hernández, Ismael Sagredo Olivenza

Colaborador

Convocatoria: Junio 2026

Calificación Alejandro Barrachina Argudo: 9.8

Calificación Pablo Sánchez Martín: 9.8

Grado en Ingeniería Informática y Grado en Desarrollo de
Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

30 de agosto de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
a los que nos han apoyado y a los que nos han
hecho crecer.
A todos los que han estado a nuestro lado.*

Agradecimientos

A mis padres y mi pareja, por estar siempre a mi lado y apoyarme en todo momento. A mi gato Umbreon, por ser mi compañero de programación, y por haber escrito casi más líneas de código que yo sentándose en el teclado. A mis juntas anteriores de LAG, por acompañarme estos años. A la gente que conocí por LAG, por haberme apoyado en los proyectos de la asociación. A Poletti, Pascal y Blanca, por haberme dado la oportunidad de hacer charlas en la facultad, de desarrollar mis conocimientos más allá de las clases y poder ayudar a más personas a buscar más conocimiento. A la gente de cafetería, por cuidarnos tan bien a todos. Y a todos los voluntarios que vinieron a hacer las pruebas para generar datos, sin ellos este trabajo no hubiera obtenido los resultados que se han conseguido.

Alejandro Barrachina Argudo

A mi madre, mi hermana, mi pareja, mi cuñado, mis tíos y mis primos por haberme apoyado y ayudado a seguir adelante. A mi grupo del instituto por estar siempre ahí para lo que necesitase. A la gente que he conocido en la universidad: tanto LAG como mi clase, por hacer que mis días en la facultad hayan sido más llevaderos. A la gente de cafetería, por cuidarme tanto en estos años. Al despacho 409 (y allegados) por soportarme todos los días, ser mis compañeros de cafetería y ayudarme cuando lo he necesitado.

Pablo Sánchez Martín

Resumen

Captura de movimientos para la detección de gestos en entornos virtuales

Este estudio se centra en la comparativa de distintos modelos de [IA](#) para la detección de gestos específicos mediante el traje de captura de movimiento Perception Neuron 3 con el fin de crear interacciones con [NPCs](#) a través de la comunicación no verbal y la creación de herramientas derivadas de los modelos que ayuden a la comunicación no verbal en entornos virtuales.

Para ello primero se buscaron datasets públicos de gestos, no encontrándose ninguno que se adaptara a las necesidades del estudio. Por ello se creó una herramienta para traducir animaciones de Mixamo a [CSVs](#) para que pudieran ser entendidas por los modelos a entrenar. Esta herramienta generó un gran número de animaciones pero muy desbalanceadas en el número de animaciones de cada tipo.

Para solucionar el problema del desbalance, se realizó una serie de pruebas con usuarios (N=65) en las cuales se les pidió que realizaran 3 tomas de cada gesto mientras llevaban el traje de captura de movimiento puesto. Esta recolección de datos resultó en un total de 975 animaciones, habiendo un total de 195 gestos humanos de correr, saludar, señalar, pegar y sentarse. Se omitió el gesto de baile en las pruebas por el desbalanceo en el dataset generado por ordenador. Esta omisión sirvió para equilibrar más el número de animaciones, pero no para terminar del todo con el desbalanceo en los datos estandarizados.

Una vez se tuvo el dataset, se implementaron distintos modelos de [IA](#) para la detección de gestos bajo una interfaz web para su facilidad de uso. Estos modelos fueron: [LSTM](#), [CNN](#), [RNN](#) y [Random Forest](#). Tras realizar los entrenamientos de todos los modelos, se observó que el modelo [Random Forest](#) era con diferencia el que mejores datos obtenía, con un 95 % de precisión en entrenamiento y un 86 % en test. Los resultados de los modelos basados en redes neuronales fueron bastante menores, pudiendo ser esto así por la falta de datos, de tiempo de entrenamiento y de hardware suficiente para explotar mejor los modelos.

Finalmente, se exportó el modelo [Random Forest](#) a TensorFlow Serving para posteriormente ser utilizado en una aplicación de demostración con un [NPC](#) reactivo. Esta aplicación permite al usuario ver como el [NPC](#) interpreta distintos gestos y responde de manera simple ante ellos.

Palabras clave

Captura de movimiento, Unity, Inteligencia Artificial, Comunicación no verbal, Perception Neuron, TensorFlow, YDF, Keras, Realidad Virtual

Abstract

Motion capture for gesture detection in virtual environments

This study focuses on comparing different [AI](#) models for detecting specific gestures using the Perception Neuron 3 motion capture suit, aiming to create interactions with [NPCs](#) through non-verbal communication and to develop tools derived from the models that enhance non-verbal communication in virtual environments.

To achieve this, public gesture datasets were first sought, but none were found that met the study's needs. Therefore, a tool was created to convert Mixamo animations into [CSVs](#) format so they could be understood by the models to be trained. This tool generated a large number of animations, but they were highly imbalanced in the number of animations for each type.

To address the imbalance issue, a series of user trials (N=65) were conducted, where participants were asked to perform three instances of each gesture while wearing the motion capture suit. This data collection resulted in a total of 975 animations, with 195 human gestures for running, greeting, pointing, punching, and sitting. The gesture of dancing was omitted from the trials due to the imbalance in the dataset generated by computer animations. This omission helped balance the number of animations but did not completely eliminate the imbalance in the standardized data.

Once the dataset was established, various [AI](#) models for gesture detection were implemented under a web interface for ease of use. These models included: [LSTM](#), [CNN](#), [RNN](#), and [Random Forest](#). After training all the models, it was observed that the [Random Forest](#) model significantly outperformed the others, achieving 95% accuracy in training and 86% in testing. The results from the neural network-based models were considerably lower, likely due to insufficient data, limited training time, and hardware constraints that hindered better model performance.

Finally, the [Random Forest](#) model was exported to TensorFlow Serving to be used in a demonstration application featuring a reactive [NPC](#). This application allows users to see how the [NPC](#) interprets different gestures and responds simply to them.

Keywords

Motion capture, Unity, Artificial Intelligence, Non-verbal communication, Perception Neuron, TensorFlow, YDF, Keras, Virtual Reality

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
2. Estado de la Cuestión	5
2.1. Comportamientos de NPCs en videojuegos	5
2.1.1. Máquinas de estado finitas	5
2.1.2. Máquinas de estado finitas jerárquicas	6
2.1.3. Árboles de comportamiento	8
2.1.3.1. Behavior Trees en Unreal Engine	9
2.2. Modelos de lenguaje	10
2.3. Generación automática de Behavior Trees	11
3. Descripción del Trabajo	13
3.1. Modelo propuesto	13
3.2. Descripción del entorno	14
3.3. Generación de los Behavior Trees	17
3.4. Validación automática de los Behavior Trees	23
3.5. Pruebas con usuarios	28
4. Conclusiones y Trabajo Futuro	37
4.1. Conclusiones	37
4.1.1. Conclusiones de los escenarios implementados	38
4.1.2. Conclusiones de la herramienta generadora de Behavior Trees	38
4.1.3. Conclusiones de la validación automática	39
4.1.4. Conclusiones de las pruebas de usuarios	40
4.1.5. Limitaciones	41
4.2. Trabajo Futuro	41
Introduction	43
4.3. Motivation	43

4.4. Objectives	44
4.5. Work Plan	44
Conclusions and Future Work	47
4.6. Conclusions	47
4.6.1. Dataset Search Conclusions	47
4.6.2. Conclusions of the Animation Extraction Tool	47
4.6.3. Conclusions of the User Data Collection	48
4.6.4. Conclusions of the IA Models Comparison	48
4.6.5. Final Application Conclusions	48
4.6.6. Limitations	48
4.7. Future Work	49
Bibliografía	51
A. JSON Schema para los Behavior Trees generados	55
B. Prompt del sistema para el LLM generador del pseudocódigo	59
C. JSON de dependencias de tareas en un Behavior Tree	61
D. Formulario de opiniones de la herramienta	63
Glosario	69
Licencia de uso del documento	71
Licencia de uso del código fuente	73

Índice de figuras

1.1. Diagrama de Gantt del proyecto	3
2.1. Ejemplo de un FSM para videojuegos	6
2.2. Ejemplo de una HFSM en videojuegos	7
2.3. Captura del sistema de animaciones de Unity Engine	7
2.4. Ejemplo de Behavior Tree	8
2.5. Ejemplo de Behavior Tree de Unreal Engine	10
3.1. Diagrama de la herramienta propuesta	13
3.2. Entorno de desarrollo	14
3.3. Generación mediante subtarefas	19
3.4. Ejemplo del pseudocódigo que el LLM debe generar.	20
3.5. Chain-of-Thought del LLM que genera el pseudocódigo.	22
3.6. Ejemplo del input y el output del LLM que establece las variables de la Blackboard.	24
3.7. Captura de pantalla del segundo entorno	27
3.8. Captura de pantalla del tercer entorno	28
3.9. Resultados de la encuesta SUS	30
3.10. Comparación de los tiempos de generación de los Behavior Trees	31
3.11. Opinión de los usuarios sobre los tiempos de generación de los Behavior Trees	31
3.12. Opinión de la calidad de los Behavior Trees	32
3.13. Opinión de los usuarios sobre la ejecución esperada de los Behavior Trees	33
3.14. Opinión sobre si los Behavior Trees son un buen punto de partida	34
3.15. Opinión de los usuarios sobre si los resultados del Behavior Tree generado son fácilmente modificables	34
3.16. Opinión de los usuarios sobre si disminuye la carga de trabajo de los diseñadores.	35
4.1. Gantt chart of the project	45
B.1. Prompt del sistema para el LLM generador del pseudocódigo	60

D.1. Formulario de recogida de datos demográficos (primera parte)	64
D.2. Formulario de recogida de datos demográficos (segunda parte)	65
D.3. Formulario de recogida de datos demográficos (tercera parte)	66
D.4. Formulario de recogida de datos demográficos (cuarta parte)	67

Índice de tablas

3.1. Modelos probados para la generación directa del JSON mediante guidance.	18
3.2. Resultados de las pruebas de validación	28

Introducción

*“La revolución industrial y sus consecuencias han sido un
desastre para la raza humana”*
— Theodore Kaczynski

En el desarrollo de videojuegos es fundamental el diseño e implementación del comportamiento de los diferentes agentes presentes en el juego, tanto los [NPCs](#) como los enemigos, con el fin de crear un mundo inmersivo para el jugador. En los equipos de desarrollo es el diseñador quien especifica los comportamientos buscados, mientras que el programador tiene la tarea de implementar las acciones requeridas. Finalmente las acciones se deben juntar en un modelo ejecutor que sea capaces de procesarlas. Este trabajo se centra en el modelo de [Behavior Trees](#).

MOVER A MOTIVACIÓN: Gracias a los avances tecnológicos, los juegos son cada vez más grandes y con sistemas más complejos, lo que ha aumentado también la complejidad de los comportamientos generados, lo que también ha causado el incremento de la complejidad de los [Behavior Trees](#). Es por esto que el desarrollo de herramientas que permitan automatizar el montaje automático de estos modelos conseguirá reducir el tiempo del desarrollo de videojuegos.

1.1. Motivación

Como se ha mencionado en la introducción, el uso de la comunicación no verbal en entornos virtuales está limitado y prefijado por el diseño de éstos, limitando una parte fundamental de la forma de expresarse cotidiana de los humanos. Debido a esto y como podemos ver en artículos como [Neverova et al. \(2013\)](#) o [Herbert et al. \(2024\)](#) la detección de gestos es un campo de estudio que actualmente se está explorando para intentar resolver este problema. Por estas razones es interesante investigar formas de ampliar esta representación de la comunicación no verbal para que la comunicación entre usuarios y [NPCs](#) sea más natural.

Es por este motivo que este trabajo se inscribe dentro de la línea de investigación de herramientas que puedan reconocer gestos realizados por un usuario mediante un traje de captura de movimiento con el fin de crear entornos virtuales más inmersivos y naturales para los usuarios.

Por otra parte, es necesaria una gran cantidad de datos para crear modelos de IA que permitan crear las herramientas mencionadas. En el caso de este proyecto, los datos necesarios son una secuencia de coordenadas 3D de posiciones y rotaciones de los huesos del esqueleto a lo largo de las animaciones, por lo que los datos que se necesitan procesar tienen un tamaño relativamente grande. Es por esto que es necesario crear modelos complejos que procesen un número significativo de datos de entrada, pero como desarrollaremos a lo largo del trabajo, no ha sido posible encontrar datasets de este tipo de datos. Como consecuencia de este motivo, una gran aportación de este trabajo en el campo ha sido la creación de un dataset gracias a la extracción de animaciones de un banco especializado en estas y a la captura de movimientos de usuarios.

Vistos estos motivos, se puede concretar que, la carencia de métodos naturales de explotar la comunicación no verbal en entornos virtuales y la falta de datasets de animaciones han motivado los objetivos mencionados y que expandiremos a continuación.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de IA que, con baja latencia, permita identificar el gesto que se esté realizando con un traje de captura de movimiento en tiempo real, con la intención de mejorar la comunicación no verbal en entornos virtuales. Para cumplir el objetivo, se han propuesto las siguientes metas durante el desarrollo del trabajo:

1. Búsqueda de datasets de animaciones que se han considerado relevantes a la hora de comunicarse con un NPC en un videojuego. Estas animaciones han sido bailar, saludar, señalar, sentarse, pelear y correr.
2. Extracción automática de las animaciones mencionadas anteriormente de bancos de animaciones y posterior procesamiento.
3. Extracción de las animaciones mediante la captura de movimiento de un grupo heterogéneo de usuarios.
4. Implementación, entrenamiento y comparación de distintos modelos de IA, tanto redes neuronales (LSTM, CNN y RNN) como modelos de clasificación clásicos (Random Forest)
5. Implementación de una aplicación final a modo de demostración para enseñar los resultados en VR para aumentar el grado de inmersión.

Como gran objetivo secundario y debido a la falta de datasets de animaciones se ha propuesto publicar los datasets resultantes, tanto de las animaciones artificiales como de las capturas de usuarios.

1.3. Plan de trabajo

El plan de trabajo consiste en varios pasos:

1. Búsqueda de un dataset: generar un dataset lo suficientemente grande con varios ejemplos de gestos como para poder entrenar de forma adecuada diferentes modelos. Este dataset debe estar compuesto en su mayoría por datos de usuarios reales para poder identificar gestos lo más naturales posibles.
2. Implementación de modelos de IA: implementación de varios modelos de IA para poder hacer una comparativa entre ellos y decidir cuál es el más adecuado teniendo en cuenta su velocidad de predicción y su precisión. La implementación de dichos modelos debe contener: entrenador, forma de sacar las estadísticas del modelo final, búsqueda de hiperparámetros, generador de matriz de confusión y debe ser acoplable a una interfaz de usuario común a todos los entrenamientos.
3. Desarrollo de una interfaz de usuario para el entrenamiento y monitorización de los distintos modelos: desarrollo de una interfaz web para que los usuarios con menos experiencia en programación e IA puedan entrenar modelos capaces de predecir los gestos que necesiten sin necesidad de modificar el código ni tener que saber como funciona internamente la base de código.
4. Desarrollo de una aplicación final: desarrollo de una aplicación para las Oculus Quest a forma de demo en la que se conecte al modelo elegido y se pueda ver en tiempo real su uso.

El desarrollo de las tareas y su planificación se puede ver en la figura 1.1.

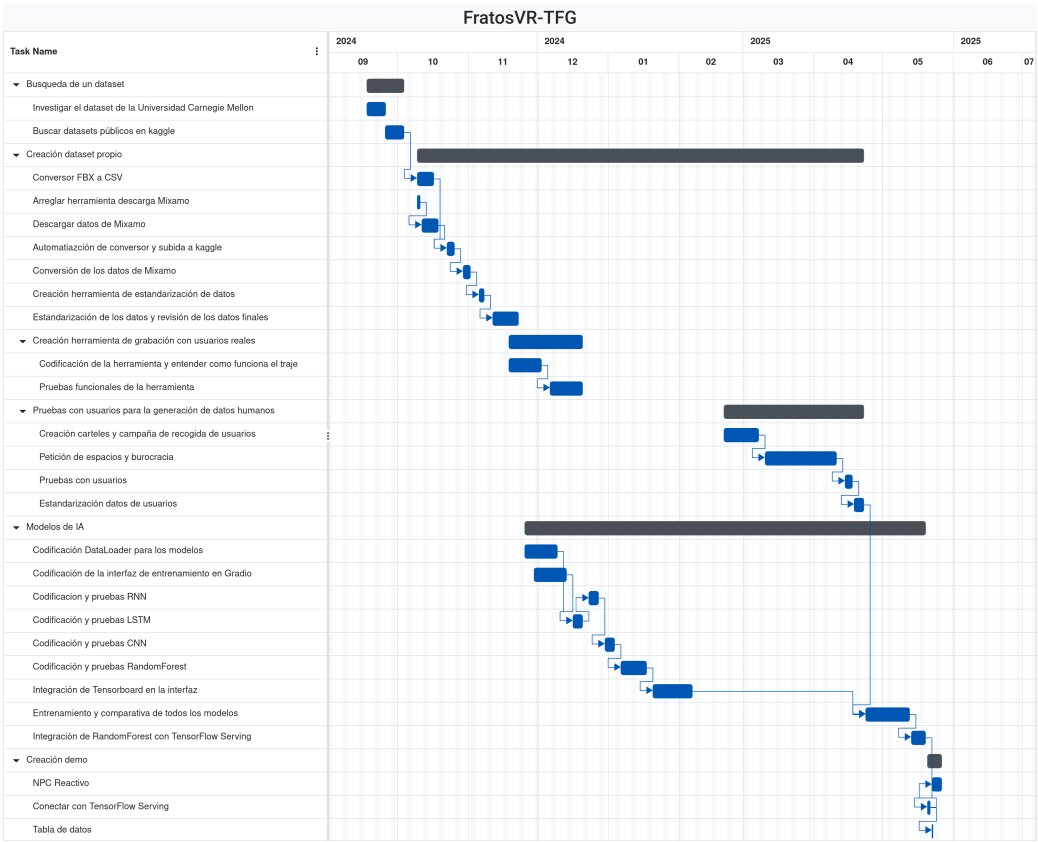


Figura 1.1: Diagrama de Gantt del proyecto

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta el estado del arte de los distintos elementos que componen el trabajo. Estos campos son los comportamientos de **NPCs** en videojuegos, los modelos de lenguaje y la generación de árboles de comportamiento mediante modelos de lenguaje

2.1. Comportamientos de **NPCs** en videojuegos

El modelado de comportamientos de **NPCs** en videojuegos ha sido siempre un aspecto fundamental en el desarrollo de videojuegos. Como bien se ha mencionado en la sección (INTRODUCIR REFERENCIA A INTRODUCCIÓN), a medida que los videojuegos han ido creciendo y evolucionando se han tenido que utilizar diferentes técnicas para modelar comportamientos cada vez más grandes y complejos, como se puede ver en [Gajardo et al. \(2019\)](#).

2.1.1. Máquinas de estado finitas

Las máquinas de estados finitas (FSM según las siglas en inglés de Finite State Machine) se definen según [Ben-Ari y Mondada \(2018\)](#) como un conjunto de estados s_i y un conjunto de transiciones s_i, s_j que están definidas por una relación de tipo acción/condición, donde la condición actúa como el evento desencadenante para realizar la transición y la acción representa la tarea específica que se ejecuta al cambiar de estado. Según su definición se puede ver que resulta muy útil para modelar el comportamiento en los videojuegos, como se puede ver en la figura ??, ya que se pueden ver a los **NPCs** como agentes que realizan acciones (permanecen en estados) constantemente, siendo las transiciones la terminación de las acciones (cuando pasan a un estado siguiente) o un factor externo.

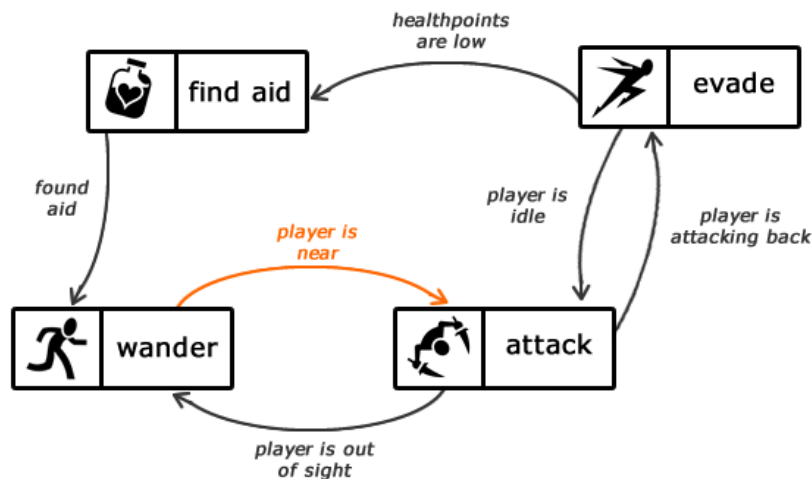


Figura 2.1: Ejemplo de un comportamiento típico de un NPCs mediante una FSM. Fuente: <https://code.tutsplus.com/es/maquinas-de-estados-finitos-teoria-e-implementacion--gamedev-11867t>

Esto se puede ver en trabajos como Fathoni et al. (2020), donde los autores presentan un videojuego en el que implementan las FSM en diferentes NPCs para que muestren un comportamiento más consistente, realista y reactivo por su simplicidad a la hora de desarrollar, dando como resultado un comportamiento inteligente y autónomo. Otro ejemplo del uso histórico de las FSMs en videojuegos se puede ver en Lee (2014), donde los autores implementan una FSM con probabilidad de transiciones distinta a 1 para emular de forma más realista el comportamiento de caza de los lobos, logrando así una experiencia más realista e inmersiva para el usuario. Sin embargo, las FSM no están tan extendidas actualmente, ya que autores como Hu et al. (2011) están de acuerdo en que las FSM tienen un problema de reutilización, flexibilidad y escalabilidad con comportamientos complejos. Aún así, actualmente se siguen utilizando como se puede ver en trabajos recientes como Ganapathi et al. (2024), donde implementan FSM de forma clásica para NPCs o en KimDongju y KohSeok-Joo (2025), donde combinan FSM con técnicas de Machine Learning como Q-Learning y DQN para controlar NPCs de un RPG.

2.1.2. Máquinas de estado finitas jerárquicas

Las máquinas de estado finitas jerárquicas (HFSM por las siglas en inglés de Hierarchical Finite State Machines) se define en Yannakakis (2001) como una extensión de una FSM cuyos estados puedan ser otras FSM contenidas (llamados “superestados”). Aunque todas las HFSM se pueden convertir en FSM mediante un proceso de *flattening* reemplazando de forma recursiva los superestados por sus máquinas internas, las HFSM han solucionado problemas de reutilización de componentes y simplicidad descriptiva. Esta reutilización ha permitido que se extendiese su uso en videojuegos, como mencionan los autores en el trabajo citado anteriormente Hu et al. (2011), con un rendimiento ligeramente mejor o similar a las FSM, como pudieron comprobar los autores en una comparativa realizada en Fauzi et al. (2019). En la

figura 2.2 se muestra un ejemplo de una HFSM en videojuegos.

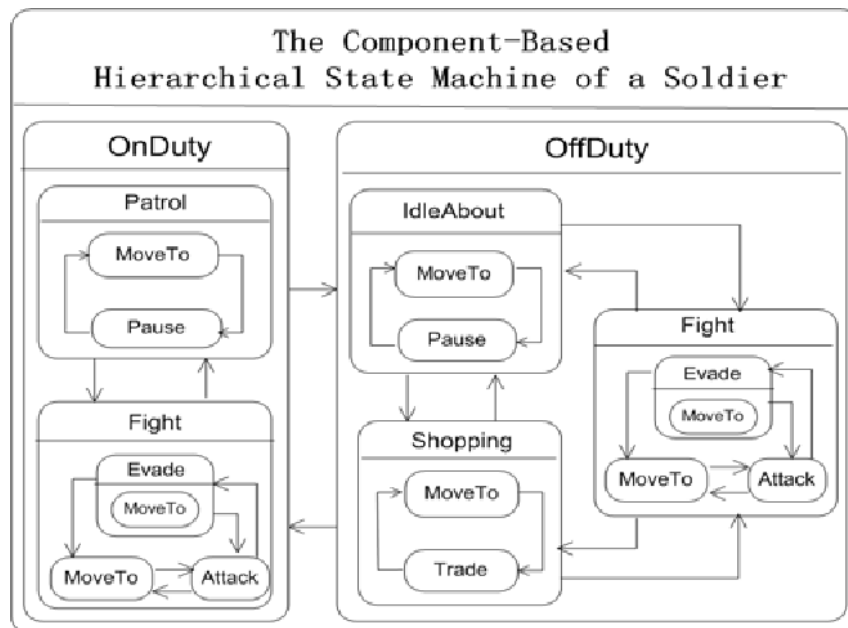


Figura 2.2: Ejemplo de un comportamiento de un NPC modelado en una HFSM. Fuente: Hu et al. (2011).

De forma comercial también se han usado las HFSM, como se puede ver en Thompson (2018). Otra forma de uso de las HFSM en videojuegos no es para comportamientos de NPCs, sino para simplificar el sistema de animaciones de videojuegos, como se puede ver en trabajos como Nur Fitriani Dewi et al. (2023). El motor de videojuegos Unity Engine también estructura su sistema de animaciones¹ en forma de HFSM para la simplicidad del usuario, como se puede apreciar en el estado de color naranja (estado “Idle”) de la figura 2.3.

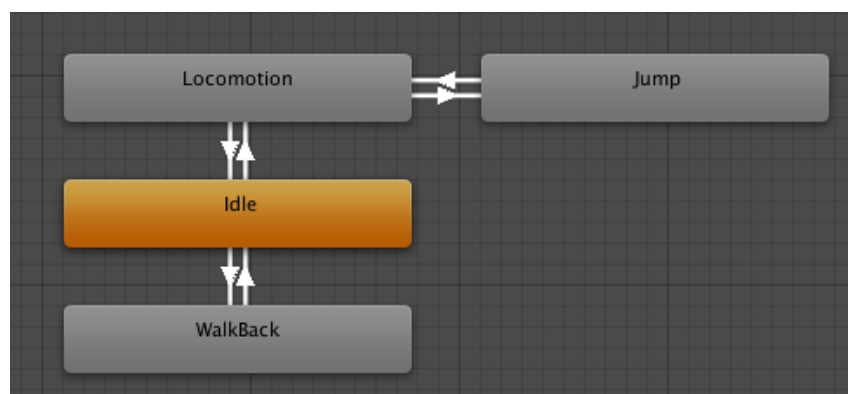


Figura 2.3: Captura del sistema de animaciones de Unity, estructurado en forma de una HFSM en la que los nodos naranja son “superestados”. Fuente: <https://docs.unity3d.com/560/Documentation/Manual/StateMachineBasics.html>

¹Enlace a la documentación de Unity: <https://docs.unity3d.com/6000.7/Documentation/Manual/AnimationStateMachines.html>

A pesar de aliviar parcialmente el problema de la duplicidad de código, autores como Sekhavat (2017) argumentan que las HFSM siguen manteniendo problemas de escalabilidad y reutilización en videojuegos; y en el trabajo de Iovino et al. (2025) los autores comentan que sigue siendo difícil de escalar.

2.1.3. Árboles de comportamiento

Los árboles de comportamiento (conocidos como Behavior Trees) se definen en Colledanchise y Ogren (2016) como árboles dirigidos formados por dos tipos de nodos: nodos hoja y nodos de composición en las que las transiciones no son unidireccionales (a diferencia de las FSM y las HFSM), sino llamadas de función bidireccionales en las que el nodo padre envía una señal periódica de activación y el hijo le responde con tres posibles estados: “Success”, “Failure” o “Running”. Un ejemplo de Behavior Tree lo podemos encontrar en la figura ??.

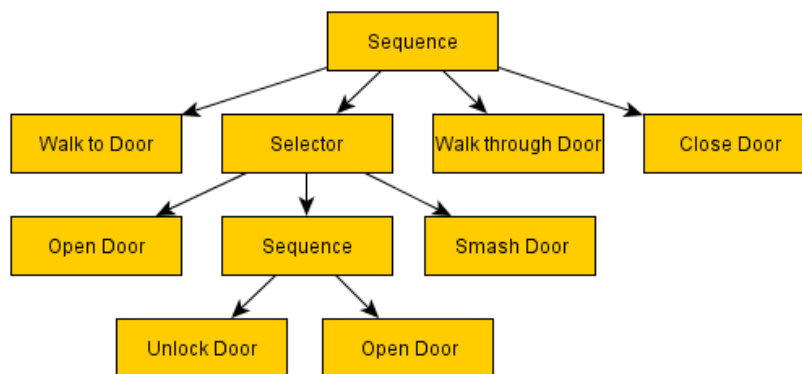


Figura 2.4: Ejemplo de un Behavior Tree para un NPC que debe pasar por una puerta. Fuente: <https://outforafight.wordpress.com/2014/07/15/behaviour-behavior-trees-for-ai-dudes-part-1/>

Como podemos ver en Iovino et al. (2022), los Behavior Trees surgieron en el mundo de la programación de IA de los videojuegos (estableciendo el videojuego Halo 2 como hito histórico) como forma de solucionar los problemas de escalabilidad, acoplamiento, adaptabilidad y reutilización que presentaban las FSM. Además, el mismo artículo comenta que este modelo resultó ser muy valioso también para la robótica, como menciona también la revisión de Iovino et al. (2020) o en trabajos como Ögren y Sprague (2022). Actualmente hay bastante literatura sobre Behavior Trees, sus implementaciones y posibles mejoras. Por ejemplo, el trabajo de Agis et al. (2020) nos habla de Behavior Trees que no son evaluados cada tick sino cuando sucede un evento (llamados Event-Driven Behavior Trees o EDBT), mejorando así la cooperación en entornos multi-agente. También podemos ver en el trabajo de Guo y Hou (2025) varias ampliaciones a los Behavior Trees en videojuegos, como los EDBT, los árboles de comportamiento emocionales (EmBT) que altera dinámicamente la prioridad de los nodos hijos dependiendo de pesos emocionales (aportando así más humanidad a los NPCs) y la evolución de los comportamientos (Evolving

Behavior), **Behavior Trees** que utilizan la programación genética para automatizar su optimización.

Aunque artículos como Tremblay et al. (2026) hablan del uso de los **Behavior Trees** como herramienta estándar de la industria (sin dejar de usar otros métodos como las ya mencionadas FSM), artículos como Gugliermo et al. (2024) mencionan que no existe un consenso sobre la descripción de sus propiedades ni sobre su semántica. Es por ello que este trabajo se va a basar en los **Behavior Trees** de Unreal Engine.

2.1.3.1. **Behavior Trees** en Unreal Engine

Unreal Engine se ha convertido en uno de los principales motores de la industria de los videojuegos, así como en el ámbito de la investigación con **Behavior Trees**, como se puede ver en trabajos como Partlan et al. (2022). Los **Behavior Trees** de Unreal Engine están basados en EDBT, como se menciona en la documentación oficial de Unreal Engine², incrementando de esta manera el rendimiento del juego.

Esto se hace gracias al uso de la **Blackboard**, un componente de memoria que almacena datos y variables de entrada/salida que usa el **Blackboard** para ciertas tomas de decisiones de ejecución.

Los tipos de nodos que proporciona Unreal se dividen en dos tipos: las composiciones y las tareas (a excepción de otros tipos de **Behavior Trees** que también consideran los condicionales como nodos adicionales). Los nodos de tipo tarea son los nodos hoja del árbol y los que aportan la lógica del NPC. Cuando se evalúan estos nodos pueden devolver el estado de “Success”, “Failure”, “In Progress” o “Aborted”.

Los tipos compuestos son nodos que controlan el flujo de ejecución del **Behavior Tree**:

- **Selector**: Ejecuta el primer hijo que cumpla con ciertas condiciones.
- **Sequence**: Ejecuta todos sus hijos en orden o hasta que uno devuelva “Failure”.
- **Simple Parallel**: Tiene dos conexiones: la tarea principal y la rama secundaria. Ejecuta las dos a la vez y se puede configurar para que su ejecución acabe con la tarea principal o tenga que esperar a la rama secundaria.

Como bien se ha mencionado anteriormente los condicionales, llamados “decoradores” en Unreal Engine, no son nodos, sino que se unen a los nodos para comprobar si pueden ser ejecutados o no. A los nodos también se le pueden unir “Servicios”, los cuales se ejecutan cada cierto tiempo determinado mientras su rama esté siendo ejecutada.

En la figura 2.5 se puede ver un ejemplo de un **Behavior Tree** realizado en Unreal Engine.

²Enlace a la documentación oficial de Unreal: <https://dev.epicgames.com/documentation/unreal-engine/behavior-tree-in-unreal-engine---overview?lang=en-US>

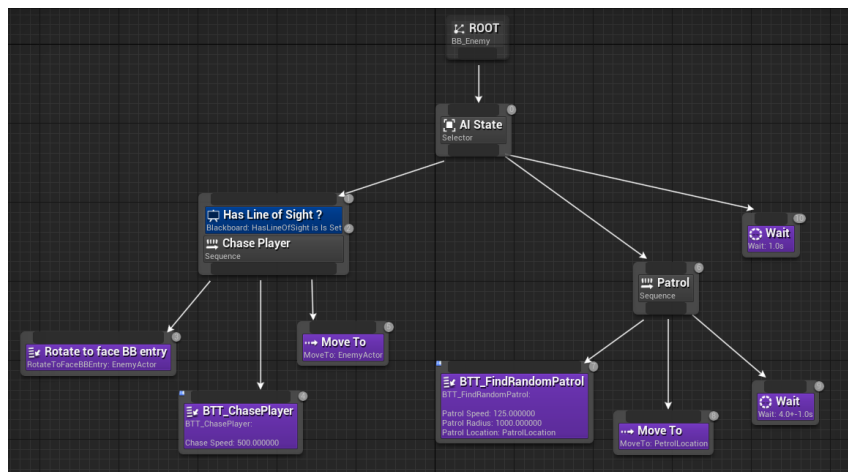


Figura 2.5: Ejemplo de un Behavior Tree de Unreal Engine de un NPC que patruya y persigue al jugador. Fuente: <https://dev.epicgames.com/documentation/unreal-engine/behavior-tree-in-unreal-engine---overview>

2.2. Modelos de lenguaje

Los modelos de lenguaje son sistemas de IA diseñados para comprender, procesar y generar texto. Al principio se basaban en modelos de lenguaje estadístico, como podemos ver en Jelinek y Mercer (1980) donde los autores cuentan como gracias al análisis de corpus de textos y el uso de n-gramas (una instancia específica de un modelo de Markov) se conseguían estimar las probabilidades de una palabra gracias a un contexto limitado. Aunque posteriormente surgieron trabajos como Berger et al. (1996), donde usaban el concepto de máxima entropía para que los modelos sean ligeramente sensibles al contexto que mejoraban estos modelos probabilísticos, seguían estando muy limitados por su baja capacidad de contexto y nulo razonamiento detrás de las respuestas.

En el trabajo de Bengio et al. (2003) se introduce un modelo de lenguaje probabilístico neuronal con el fin de aprender una representación distribuida para cada palabra del vocabulario asociándola con un vector continuo de características reales y una función de probabilidad condicional de la siguiente palabra dependiendo de su contexto, consiguiendo una reducción de la perplejidad (métrica de los modelos de lenguaje que mide la precisión de predicción de la siguiente palabra) de entre el 10 % y el 20 % y permitiendo aprovechar contextos de entrada más largos y más distribuidos, no solo teniendo en cuenta las N palabras anteriores. Este trabajo permitió una evolución de modelos de lenguaje probabilísticos a los basados en redes neuronales. En Cho et al. (2014) se introduce el concepto de las redes neuronales recurrentes (RNN por las siglas en inglés de Recurrent Neural Network) para su uso en traducción de textos.

LLMs, CHATs Y APIs

PROMPT ENGINEERING

AGENTES Y MCP

2.3. Generación automática de Behavior Trees

EJEMPLOS DE GENERACIONES ANTERIORES A LLMs

GENERACIÓN CON LLMs EN ROBÓTICA

GENERACIÓN CON LLMs EN VIDEOJUEGOS.

Capítulo 3

Descripción del Trabajo

3.1. Modelo propuesto

Esta herramienta debería ser de fácil uso para los diseñadores y programadores, por lo que hemos propuesto un modelo en el que, sin salir del proyecto de Unreal Engine, puedan generar los Behavior Trees de forma sencilla.

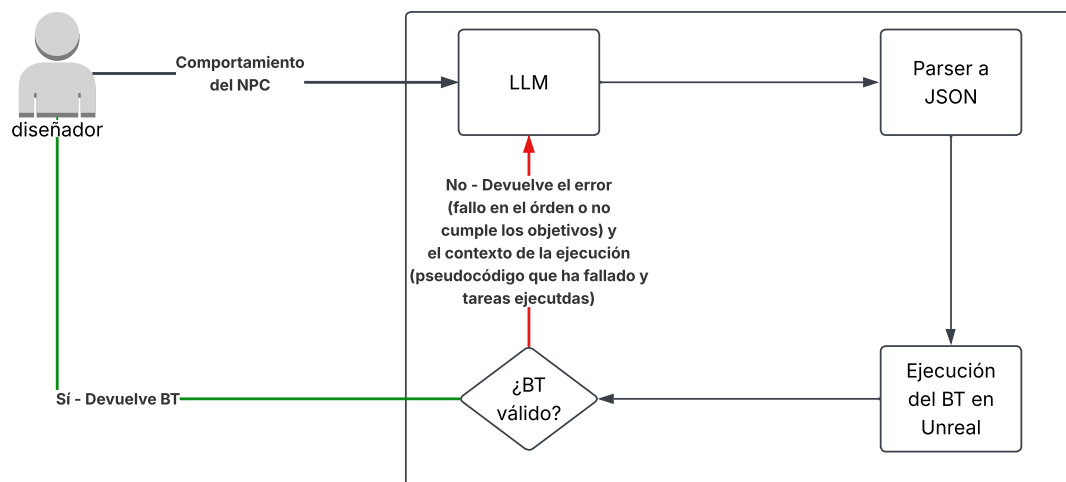


Figura 3.1: Diagrama donde se visualiza el flujo de la herramienta propuesta.

En la figura 3.1 se puede ver el flujo de la herramienta, donde el diseñador describe el comportamiento que busca para un NPC y, de forma opaca para éste, un LLM genera el Behavior Tree, el cual deberá ser parseado a formato JSON. Una vez creado el BT, se devuelve el control a Unreal Engine, donde se ejecutará de forma automática para comprobar que es un Behavior Tree válido. En caso de ser así se termina la ejecución de la herramienta, pero en caso contrario se reinicia su ejecución, donde se le pasa al LLM el error del Behavior Tree generado para volver a intentarlo.

En las siguientes secciones de este capítulo se explicará el entorno de pruebas

desarrollado para este trabajo, así como la implementación de la generación de los [Behavior Trees](#), la validación de éstos y las pruebas realizadas.

3.2. Descripción del entorno

Para poder desarrollar la herramienta se han creado unos entornos de pruebas basados en Unreal Engine 5.7. Son estos entornos los que se van a comunicar con el agente [LLM](#) de Python.

Estos entornos, de los que se pueden ver un ejemplo en la figura 3.2, constan de un agente (el [NPC](#) que va a seguir el comportamiento buscado) cuyo controlador es una clase que hereda de la clase de Unreal Engine “[AIController](#)”. Esta clase contiene los componentes de [Blackboard](#) y [Behavior Tree](#) necesarios para la futura ejecución de un [Behavior Tree](#). En la misma figura se puede apreciar la interfaz en la que el usuario debe introducir la información necesaria para la generación del [Behavior Tree](#): la descripción del comportamiento buscado, el nombre con el que se va a guardar el [Behavior Tree](#) generado, las entradas de la [Blackboard](#), el agente que va a ejecutar el [Behavior Tree](#) generado y, opcionalmente, el nombre del archivo de validación del [Behavior Tree](#) generado. Esta información la debe contener un actor, llamado en la captura “[BTGeneratorManager](#)”, que tenga un componente llamado “[BTGenerator](#)”. Este componente de actor es el encargado de lanzar el proceso de Python con la información que el usuario haya puesto, y el que comienza la ejecución de las pruebas del [Behavior Tree](#) generado. Para comenzar la ejecución se debe pulsar el botón “[GenerateBT](#)” que se aprecia en la captura.

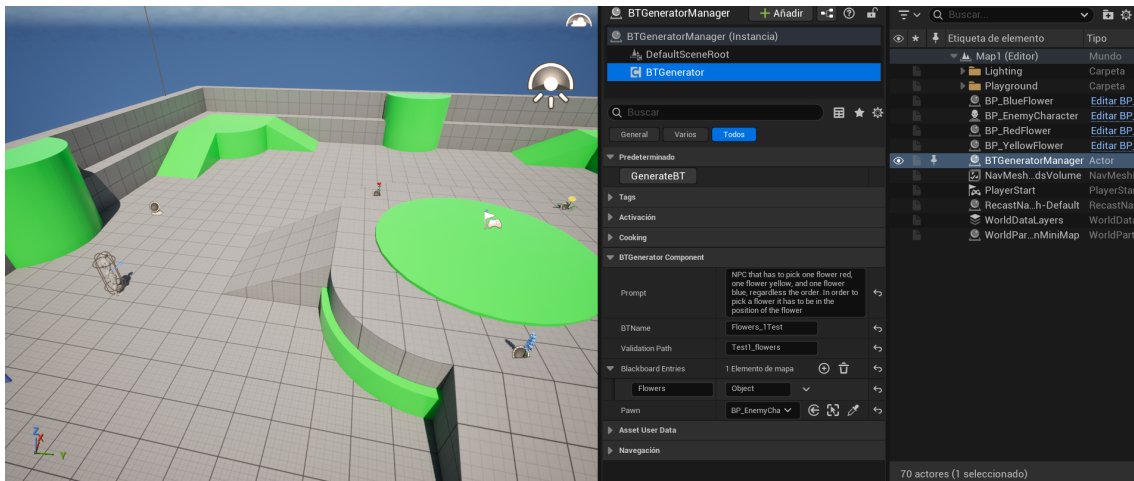


Figura 3.2: Captura de pantalla de uno de los entornos creados para el desarrollo de la herramienta.

Junto con el agente mencionado anteriormente se han preparado una serie de tareas (los cuales servirán como nodos terminales de los árboles) y condicionales que sirven como base para poder crear unos comportamientos de prueba. Estas tareas y condicionales son introducidos en una clase “[BTFactory](#)”, la cual es un [Abstract Factory](#) implementado para la construcción de los [Behavior Trees](#). Esta clase, la cual

se inicializa desde el componente “BTGeneratorComponent”, contiene tres diccionarios que guardan las tareas, las tareas que necesitan una variable de la [Blackboard](#) y los condicionales. Estos diccionarios reciben como clave un string, que es la ID de la tarea o el condicional, y como valor una función (implementada en la propia clase) que crea el objeto que se ha pedido. Además esta clase tiene otro diccionario que contiene la descripción de cada tarea y condicional, la cual es fundamental para la futura generación de los [Behavior Trees](#). Para registrar una nueva clase en esta factoría se debe llamar a las funciones “RegisterTask”, “RegisterBlackboardTask” o “RegisterDecorator”, las cuales son funciones template en las que se les debe poner la clase que se desea ser introducida y pasar como argumentos el ID que se desee poner y la descripción de lo que realiza la tarea o lo que revisa el condicional. Además, para las clases que requieran una variable de la [Blackboard](#) es recomendable introducir qué tipo de variable se requiere.

Las tareas, junto con sus descripciones en español (en el trabajo realizado las descripciones han sido escritas en inglés para respetar el idioma del prompt) que se han creado para estas pruebas son las siguientes:

- **RotateToFaceBBEntry**: Rotar hasta encarar objetivo. El objetivo tiene que ser de tipo “Objeto”.
- **ChasePlayer**: Perseguir al jugador.
- **MoveTo**: Mover a la posición u objeto de destino. La variable objetivo tiene que ser de tipo “Objeto” o “Vector3”
- **FindRandomPatrol**: Elegir un punto aleatorio del mapa y guardarlo como objetivo.
- **Wait**: No hacer nada.
- **PickRedFlower**: Coger una flor roja.
- **PickBlueFlower**: Coger una flor azul.
- **PickYellowFlower**: Coger una flor amarilla.
- **PickBlackFlower**: Coger una flor negra.
- **ChooseRedFlower**: Elegir una flor roja como objetivo.
- **ChooseBlueFlower**: Elegir una flor azul como objetivo.
- **ChooseYellowFlower**: Elegir una flor amarilla como objetivo.
- **ChooseBlackFlower**: Elegir una flor negra como objetivo.
- **Attack**: Atacar si hay algún objetivo cerca.
- **SelectWaypoint**: Elegir un punto de ruta aleatorio y establecerlo como objetivo. El objetivo debe ser de tipo “Objeto”

Los condicionales implementados, junto con sus descripciones en español (al igual que las tareas, las descripciones en el trabajo están en inglés), son los siguientes:

- **HasLineOfSight?**: Ciertamente si el jugador está en línea de visión, falso si no. La variable que verifica tiene que ser de tipo “Bool”
- **IsItNear?**: Ciertamente si la distancia entre el ejecutor y el jugador es menor o igual que un umbral prefijado, falso en caso contrario. El objetivo tiene que ser de tipo “Objeto”.

Las tareas y condicionales que necesiten una variable de la [Blackboard](#) necesitan seleccionarla desde el objeto de la [Blackboard](#) (llamado en Unreal “BlackboardData”) a través de una variable que tienen los condicionales y las tareas que hereden de la clase de Unreal “BTTask_BlackboardBase”, que son las tareas que requieren de una variable. Sin embargo esta variable es protegida, por lo que se ha creado una interfaz intermedia llamada “BaseTask” (de la que heredan todas las tareas que requieran una variable) que contiene las funciones necesarias para acceder a esta variable protegida que permite seleccionar la variable deseada de la [Blackboard](#).

El número de terminales y decoradores disponibles en esta implementación es reducido, pero sirve como sistema de prueba para la generación y la propuesta de evaluación.

Para la generación del JSON se ha seguido una definición de JSON Schema (Draft 2020-12) que delimita la estructura de la generación del JSON para adaptarlo a las necesidades del trabajo. Este JSON Schema consta de dos partes: el [Blackboard](#) y el [Behavior Tree](#). El [Blackboard](#) es una lista de objetos JSON que contienen solo un par clave-valor, donde la clave es el nombre de la variable y el valor es el tipo de la variable (Bool, Int, Float, Vector, Object o String). El [Behavior Tree](#) consta de un nodo (la raíz) que debe contener el tipo de nodo (Task, Sequence, Selector o Parallel), una lista de decoradores (en el caso de no tener se debe quedar vacía), la cual consiste en objetos JSON que debe contener como clave el nombre del decorador y como valor la variable de la [Blackboard](#) que va a usar, y una lista de nodos hijos. Estos nodos hijos siguen la misma estructura que la raíz. Como propiedades adicionales, puede contener una lista de nombres de variables de la [Blackboard](#) que vaya a necesitar, y, en el caso de ser una tarea, debe tener el ID de ésta. Este esquema se puede encontrar en el apéndice [A](#)

Para cuando el modelo haya generado el JSON con el [Behavior Tree](#), la clase “BTGeneratorComponent” entra en el modo de juego. Es en este momento donde el [NPC](#), desde su función “BeginPlay” (función de Unreal que es llamada cuando se inicia el juego), coge el nombre del archivo JSON generado para traducirlo a la lógica de Unreal. Esta traducción la hace la clase “BTConstructor”, la cual es una clase que sigue el patrón [Singleton](#) que es inicializada desde la clase “BTGeneratorComponent”. Para la traducción se le pasa a la clase el nombre del archivo y el componente de [Blackboard](#) que debe contener el [NPC](#) mediante el método “CreateBT”. Este método crea un [Behavior Tree](#) (clase “BehaviorTree”) y un [Blackboard](#) (clase “BlackboardData”) que serán rellenados con la información del JSON.

Lo primero que se traduce es el [Blackboard](#) gracias al método “CreateBlackboardAsset”, el cual recorre toda la lista de variables para, dependiendo del tipo de ésta,

crear un nuevo objeto de Unreal de tipo “BlackboardKeyType_XX” (donde las XX es el tipo de la variable). Éstas se guardan dentro del campo “KeyType” de una variable creada de tipo “BlackboardEntry”, que será agregada al campo “Keys” de la variable de tipo “BlackboardData”. Cuando ya se han creado todas las variables de la [Blackboard](#) se inicializa mediante el método “InitializeBlackboard” del componente “BlackboardComponent”.

Finalmente se crea el [Behavior Tree](#) mediante el método recursivo “CreateNode”. Inicialmente crea un objeto de tipo “FBTCompositeChild” en el que se guardará y devolverá el nodo que se le haya pasado como parámetro (inicialmente el nodo raíz). Si no es de tipo “Task” se crea un objeto de tipo “BTCompositeNode”, del cual heredan los nodos compuestos (Selector, Sequence y Parallel) para, posteriormente y de forma recursiva, inicializar los nodos hijos comprendidos dentro de la lista “Nodes” y establecérselos como sus hijos. Si por el contrario es de tipo “Task” se le pide a la clase “BTFactory” una nueva instancia de esa tarea (si no existe esta tarea se devuelve un nodo vacío). Además, si necesita alguna variable de la [Blackboard](#), se llama a las funciones necesarias de éstas tareas para establecérsela. Finalmente, tanto como para las tareas como para los nodos compuestos, se recorren sus decoradores para inicializarse de la misma forma que las tareas: pidiéndole a “BTFactory” una instancia de esa clase y estableciendo las variables de la [Blackboard](#) necesarias.

Una vez se ha especificado todo lo necesario para la generación de los [Behavior Trees](#) se ha procedido a la conexión con la herramienta que los genera. Como se ha mencionado en la sección 3.1, por comodidad del usuario es el propio entorno quien lanza el proceso de Python. Para ello se llama a la función “ExecProcess” de la clase de Unreal “PlatformProcess” con “python” como el proceso que debe ser lanzado y como argumentos la ruta del script de la herramienta, las listas de tareas (la lista de tareas que requieren de una variable de la [Blackboard](#) y la lista de las que no), la lista de decoradores, el comportamiento buscado y la lista de entradas de la [Blackboard](#).

3.3. Generación de los Behavior Trees

En un principio se planteó la generación de los [Behavior Trees](#) de forma directa, en la que se le pasaba el prompt del usuario a un modelo local y éste generaba un [Behavior Tree](#) en el formato JSON especificado en el apéndice A. Este planteamiento se realizaba mediante el módulo de Python “guidance”, el cual permite forzar un formato de salida a un modelo de [Ollama](#). Para este caso se probaron diferentes modelos, todos probados bajo el mismo escenario y con la temperatura en 0.0:

- **Prompt:** [NPC](#) that, constantly, select a random point from the map, goes to the point and waits a time.
- **Acciones disponibles:** ChooseRandomPoint, MoveTo, WaitTime, Rotate-ToFaceBBEntry.
- **Variables de la [Blackboard](#):** Point (Vector3)

Se buscó una tarea sencilla para establecer un baseline de los modelos. Los modelos probados para este planteamiento, con sus parámetros, su tiempo de ejecución y su resultado se pueden ver en la tabla 3.1:

Tabla 3.1: Modelos probados para la generación directa del JSON mediante guidance.

Modelo	Parámetros	Tiempo de ejecución
microsoft/Phi-4-mini-instruct	4B	10 segundos
qwen2.5:7b	7B	20 segundos
Qwen/Qwen2.5-Coder-7B-Instruct	7B	30 minutos
llama3.1:8b	8B	2 horas
deepseek-r1:8b	8B	>2 horas

Los modelos de menor tamaño no generaban buenos [Behavior Trees](#) para una tarea tan sencilla (como tareas que faltaban o variables de la [Blackboard](#) mal asignadas). Sin embargo, los modelos grandes si conseguían generar [Behavior Trees](#) que cumplieran con la función que se les pedía, aunque añadiendo más tareas de las necesarias. Pero, como se puede observar en la tabla, para conseguir esos [Behavior Trees](#) la ejecución podía tardar horas, por lo que también debían ser descartados. Tanto la malfuncionalidad de los modelos pequeños como la tardanza de los modelos grandes se puede explicar por el funcionamiento de guidance y la complejidad recursiva del JSON Schema empleado, por lo que esta primera generación debería ser descartada.

En base a otros trabajos relacionados como [Hoffmeister et al. \(2026\)](#) o [Choi et al. \(2026\)](#) se replanteó la generación para dividirlo en subtareas con el fin de simplificar la tarea al [LLM](#). Para ello se encargaban cuatro agentes [LLM](#), cada uno con un rol distinto. El primero se encargaba de dividir la acción mandada por el usuario mediante el prompt en subtareas. En el ejemplo de un [NPC](#) que tiene que buscar al jugador para perseguirlo y atacarlo cuando esté cerca, este primer agente debería dividir la tarea principal en:

1. Buscar al jugador
2. Perseguir al jugador
3. Comprobar la distancia del [NPC](#) al jugador
4. Atacar al jugador

Una vez están separadas las tareas en subtareas, un segundo agente selecciona para cada subtask el subconjunto de las acciones necesarias para realizar esta subtask. Estas acciones que selecciona se guardan en una lista directamente en el formato JSON especificado por el JSON Schema. Cuando ya se ha seleccionado el subconjunto un tercer agente decide qué tipo de nodo padre van a tener los subconjuntos (Selector, Sequence o Parallel). Finalmente un último agente revisa los decoradores

y las tareas que requieran de una variable de la [Blackboard](#) para adjudicársela. Se puede ver un diagrama de flujo de esta generación en la figura 3.3.

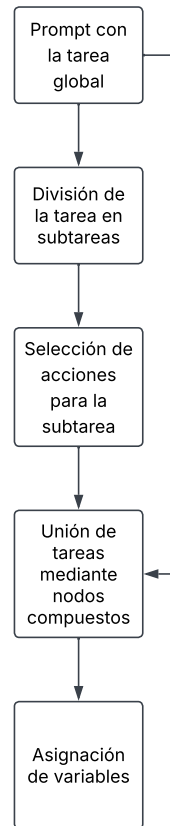


Figura 3.3: Diagrama de flujo de la generación de [Behavior Trees](#) mediante división en subtareas

Finalmente se descartó esta opción debido a que los árboles resultantes quedaban muy planos (con menos niveles de profundidad de lo que se cabía esperar), haciéndolos difícilmente legibles y modificables. Además los agentes tendían a equivocarse (mala división en subtareas, confusión entre el tipo que debería ser un nodo padre) o a alucinar (inventarse tareas o decoradores que no existen), no consiguiendo de este modo un [Behavior Tree](#) válido con ninguno de los modelos probados.

Trabajos como [Ahn et al. \(2022\)](#) o [Song et al. \(2023\)](#) muestran que, dado un conjunto de acciones posibles a realizar, los [LLMs](#) son capaces de seleccionar de éstas un orden para realizar una tarea de alto nivel. Es por ésto que se decidió que el [LLM](#) generase esta secuencia en un formato de pseudocódigo que fuese fácilmente convertible al formato JSON deseado. Este pseudocódigo se especifica de la siguiente manera:

- Las palabras reservadas son: SEQUENCE, SELECTOR, PARALLEL, IF, THEN, ELSE, DO, END.
- Los bloques de nodos compuestos (como SELECTOR) deben acabar con la palabra END.

- Las condiciones deberán ser escritas como “IF (nombre del condicional de la lista de decoradores) THEN”.
- Los condicionales serán usados por el nodo siguiente.
- Las acciones deberán ser escritas como “DO (nombre de la acción de la lista de acciones)”.

Se puede ver un ejemplo de este pseudocódigo en la figura 3.4.

```
SELECTOR
  IF has_key THEN
    SEQUENCE
      DO move_to_door
      DO open_door
    END
  ELSE
    SEQUENCE
      DO find_key
      DO move_to_door
      DO open_door
    END
  END
END
```

Figura 3.4: Ejemplo del pseudocódigo que el LLM debe generar.

Como se ha podido ver en trabajos relacionados, la generación de *Behavior Trees* mediante LLMs mejora considerablemente en modelos a los que se le ha aplicado *Fine-tuning* o se les proporciona ejemplos. Debido a la falta de capacidad computacional no se ha podido hacer un *Fine-tuning* de un LLM, por lo que se ha optado a recopilar una pila de ejemplos de *Behavior Trees* para poder hacer un sistema RAG. Para ello se ha escogido el dataset publicado por Lykov y Tsetserukou (2023)¹, el cual contiene 8366 ejemplos de *Behavior Trees*. Debido a que estos *Behavior Trees* están en formato XML no se pueden utilizar de forma predeterminada para el sistema RAG, por lo que se ha procedido a hacer un preprocesamiento de los datos. Este preprocesado carga el dataset desde Hugging Face, los limpia y los traduce al formato necesario. La limpieza consiste en el uso de expresiones regulares para poder subsanar los fallos de formato más comunes que tenían estos datos, como diferencias entre nombres por mayúsculas y minúsculas, etiquetas sin la apertura o la clausura, faltas de comillas, etc. Si hay algún ejemplo que, aun con la limpieza, no se ha conseguido parsear por otro fallo, se descarta. Finalmente, de los 8366 ejemplos se quedaron 8290 *Behavior Trees* válidos. Una vez los datos están limpios, se procede a la traducción del formato XML gracias al módulo “xml.etree.ElementTree” mediante una función recursiva que recorre el XML y va construyendo todo el árbol. Cuando el árbol ha sido construido se guarda en formato JSON siguiendo el JSON Schema

¹Enlace a la página de Hugging Face: https://huggingface.co/datasets/ArtemLykov/LLM_BRAIn_dataset

especificado para este trabajo, y finalmente se traduce al pseudocódigo especificado previamente gracias a otra función recursiva, quedando así 8366 archivos JSON con el prompt de la acción de alto nivel a realizar, el Behavior Tree resultante y el pseudocódigo.

Con los ejemplos de Behavior Trees preparados se ha procedido a la aplicación del sistema RAG. Al iniciar la herramienta de generación se crea el Almacén vectorial cargando los documentos mencionados anteriormente en estructuras llamadas “Document”, provenientes del módulo “langchain_core.documents”. Una vez están todos los documentos cargados se procede a hacer el Embedding de éstos de manera local gracias al modelo de Hugging Face “all-MiniLM-L6-v2” mediante el módulo “langchain_huggingface”. Estos Embeddings se crean y se almacenan en una estructura de tipo FAISS gracias al módulo “langchain_community.vectorstores”. Una vez creado el Almacén vectorial se rescatan las 5 búsquedas más similares al comportamiento pedido por el usuario y se incluyen en el prompt del sistema del LLM.

Para la generación de pseudocódigo a partir del comportamiento buscado se ha requerido un modelo de mayor tamaño que los previamente usados, por lo que se ha usado el LLM “Mistral Large 3”, el cual cuenta con un total de 675B de parámetros (41B de ellos son parámetros activos, los que actúan por cada token), mediante el módulo “langchain_mistralai”, el cual realiza llamadas de API al modelo. Para el prompt de sistema del modelo se han realizado varias técnicas de *prompt engineering* con el fin de reducir las alucinaciones en la generación del pseudocódigo:

- **Role prompting:** Se le especifica al LLM como debe actuar: “You are an expert in behavior trees and algorithm design.”.
- **Especificación de la tarea:** Se le especifica para qué tarea debe responder: “Your task is to convert a high-level task description into a structured pseudocode algorithm that can later be transformed into a Behavior Tree.”.
- **Few-shot prompting:** Se le especifican varios ejemplos, sacados del sistema RAG mencionado anteriormente.
- **Chain-of-Thought:** Obliga al modelo a, internamente, seguir la línea de pensamiento representada en la figura 3.5
- **Self-Critique Prompting:** Le pide al modelo que evalúe su respuesta buscando si se ha inventado acciones o condicionales. Esto se hace en una segunda llamada con el siguiente prompt: “Verify if your pseudocode is made only with the actions and conditions available. Responde me with ONLY the final pseudocode and no explanations or plain text”

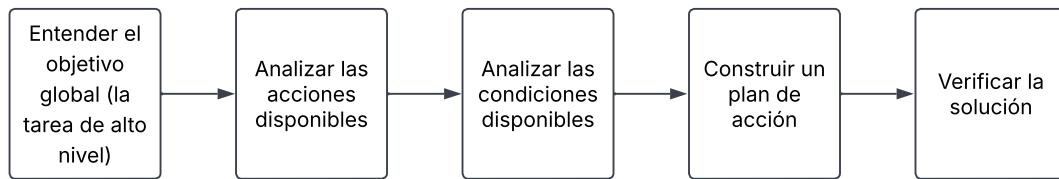


Figura 3.5: Diagrama de la cadena de pensamiento que sigue el [LLM](#) generador de pseudocódigo.

El prompt de sistema completo se puede ver en el apéndice [B](#).

Una vez se ha generado el pseudocódigo se procede a la traducción de éste al formato JSON. Esto se hace mediante la clase “BehaviorTreeParser” y mediante dos clases auxiliares: Composite y Action, las cuales ambas heredan de la clase Node. La clase Composite contiene el tipo (un string), una lista de nodos hijos (lista de tipo “Node”), un decorador (opcional de tipo string), las variables de la [Blackboard](#) del decorador (una lista de strings) y un identificador (un entero). La clase Action tiene el nombre de la tarea (un string), las variables de la [Blackboard](#) que puede usar la tarea (una lista de strings), un decorador (opcional de tipo string), las variables de la [Blackboard](#) del decorador (una lista de strings) y un identificador (un entero). Para la traducción del pseudocódigo primero se preprocesa guardando cada línea en una lista de strings. Seguido de esto se procesa cada línea dividiendo cada palabra de ésta y actuando dependiendo de cuál es su primera palabra:

- **SEQUENCE, SELECTOR o PARALLEL:** Se crea un nodo de tipo “Composite”, se establece su tipo como la palabra usada y se verifica si hay una condición pendiente. Si es el caso, se establece esta condición. Además, se añade este nodo a su padre (en el caso de haberlo es el último nodo añadido a una pila auxiliar) y se añade a la pila auxiliar.
- **IF:** Se verifica si tiene un condicional o si el condicional que tiene existe. En caso contrario de cualquiera de los dos se lanza un “ParserError” indicando la línea y el fallo. En el caso de que no haya error se establece la condición como la condición pendiente.
- **DO:** Se verifica si tiene una acción o si esa acción existe. Al igual que en el caso anterior, de no ser así se lanza un “ParserError”. En caso de haberlo se crea un nodo de tipo “Action” con el nombre de la acción que hay a continuación. Igualmente que con los nodos compuestos, se verifica si hay alguna condición pendiente y se añade el nodo al padre.
- **END:** Se verifica que la pila no esté vacía. En caso contrario significa que hay un “END” sin bloque que cierre, por lo que se lanza un “ParserError”. En caso de que sí esté vacía se saca un nodo de la pila. A continuación se verifica si la pila se ha quedado vacía, lo que significaría que ese nodo era la raíz del árbol. En caso de ya haber raíz se lanza un “ParserError” indicando que hay múltiples raíces.

Una vez se acaba este procesamiento de cada línea se verifica si sigue habiendo pila (lo que indicaría que hay bloques sin cerrar) o si no se ha generado una raíz. Los posibles “ParserError” lanzados durante este proceso se recogen fuera de esta función para poder volver llamar al LLM indicando el error de la excepción con el fin de que devuelva un pseudocódigo arreglado.

Cuando ya ha sido procesado todo el pseudocódigo se crea una clase auxiliar llamada “BlackboardAssignmentAgent” que se encarga de asignar las variables de la Blackboard a las tareas o los decoradores. Para ello reconstruye el pseudocódigo asignándole el identificador a cada nodo (este identificador es el orden de aparición de cada acción o decorador, ya que en los Behavior Trees es normal que se repitan) y se le pide a un segundo LLM que a cada decorador y acción que necesite una variable de la Blackboard se le asigne. Por problemas con sobrecarga de peticiones de la API, para este caso se usa un modelo local más pequeño, el modelo “llama3:8b”. Para el prompt de sistema de este LLM se le pasa el árbol en el formato del pseudocódigo reconstruido mencionado anteriormente, la lista de las acciones que necesitan una variable de la Blackboard con sus descripciones y la lista de variables de la Blackboard. Para disminuir las alucinaciones de este LLM se han usado las siguientes técnicas de *prompt engineering*:

- **Role prompting:** “You are an expert in Unreal Engine Behavior Trees.”
- **Especificación de la tarea:** “Your task is NOT to understand the whole game. Your ONLY task is assigning Blackboard variables to Behavior Tree nodes.”
- **One-shot prompting:** Se le especifica un ejemplo inventado mostrado en la figura 3.6

Una vez genera la respuesta se valida el resultado comprobando si todos los nombres (variables y acciones) son correctos. Cuando el resultado es válido se guardan las variables asignadas en los nodos correspondientes y, finalmente, se construye el JSON mediante una función recursiva desde la raíz del árbol y se guarda en un archivo con el nombre especificado por el usuario.

Trabajos como Valmeekam et al. (2023) han demostrado que la planificación generada por LLMs como se ha hecho en este trabajo no resultan ser fiables de forma exacta, por lo que se ha diseñado un sistema de validación y corrección de errores automático.

3.4. Validación automática de los Behavior Trees

Para que los Behavior Trees generados se considerasen válidos para las pruebas se han tenido en cuenta dos factores: que los árboles generados respeten el orden de los nodos (si hay acciones dependientes unas con otras no deben ejecutarse los hijos antes que los padres) y que deben cumplir un objetivo. Estas comprobaciones no son obligatorias para el funcionamiento de la herramienta, sino que es un apoyo para automatizar este proceso. Para comprobar el orden correcto de los nodos se ha

Example Input:

Actions : grab_key : Stores the key in the inventory,

move_to : Moves to a position,

open_door: Open a door

Variables: ["Key" : "Object", "Door" : "Object", "Enemy_position" : "Vector3"]

Tree:

SEQUENCE

DO move_to#1

DO grab_key#1

DO move_to#2

DO open_door#1 [Decorator=IsNear?#1]

DO wave#1

Output format:

```
{
  "Actions": {
    "move_to#1": ["Key"],
    "grab_key#1": ["Key"],
    "move_to#2": ["Door"],
    "open_door#1": ["Door"]
  },
  "Decorators": {
    "IsNear?#1": ["Door"]
  }
}
```

Figura 3.6: Ejemplo del input y el output del [LLM](#) que establece las variables de la [Blackboard](#).

recurrido al uso de un autómata finito no determinista (NFA por las siglas en inglés de Non-Deterministic Finite Automata) definido de la siguiente manera:

- Los estados son los nodos de [Behavior Tree](#).
- Las transiciones son las relaciones padre-hijo.
- El alfabeto son las IDs de las tareas.
- Hay un conjunto de estados abiertos.
- Hay una serie de estados iniciales.
- La función de transición es no determinista porque un mismo símbolo puede activar múltiples nodos de forma simultánea, ya que diferentes tareas pueden estar en diferentes puntos del árbol.

Este NFA está implementado en la clase “BehaviorList”, la cual es inicializada por la clase “Generator Manager”. Esta clase, implementada mediante el patrón [Singleton](#), necesita de un archivo JSON donde esté la especificación de las dependencias.

Al iniciarse esta clase se lee el archivo especificado en la clase “Generator Manager” con el JSON que especifica el orden de los nodos y se construyen las dependencias mediante la función recursiva “addTask”, la cual recibe un objeto de tipo “json” y el ID del nodo padre (en la primera llamada se establece como “INDEX_NONE”). Del tipo json se recorren todos los nodos que haya en este nivel, y a cada uno se le asigna una ID (el número de nodos que hay presentes) y se guarda su nombre y el ID de su padre en una clase auxiliar “NodeDefinition”. Cada nodo se guarda en un array y en un “TMultiMap” (mapa que cada clave puede albergar varios valores, siendo la clave el nombre de la tarea realizada y los valores sus IDs) y a su nodo padre se le adjudica el hijo (esto es gracias al array y a la nueva ID creada) para que finalmente se llame a la misma función para guardar a los posibles hijos. En el apéndice [C](#) se puede ver un ejemplo de un JSON con la dependencia de las tareas. Una vez se construyan las dependencias empieza la ejecución del [Behavior Tree](#), y para que funcione esta comprobación a la hora de ejecutarse una tarea o un decorador, éste debe llamar a la función “addBehavior” de esta clase con el nombre de la tarea. Cuando llega una tarea nueva se realiza la comprobación de las dependencias de la siguiente forma:

1. Se recuperan los nodos candidatos gracias al “TMultiMap”.
2. Se crea un nuevo conjunto de nuevos estados.
3. Si un candidato no tiene padre (no depende de ninguna otra acción) se añade este nodo a los nuevos posibles estados.
4. Se recorren todos los hijos de los estados activos y se comprueba si coinciden con el ID de la tarea o el decorador actual. En caso afirmativo se añade este hijo al conjunto de nuevos estados.

Al final de la función se comprueba si el conjunto de nuevos estados está vacío. En caso de que sí se hayan respetado las dependencias el conjunto de nuevos estados pasa a ser el conjunto de estados abiertos. Por el otro lado, si el conjunto de nuevos estados está vacío es que no existe ningún posible estado en el que el orden de la ejecución de los nodos sea la actual, por lo que no se han respetado las dependencias, por lo que la validación falla. Para los casos en los que la validación puede fallar se ha creado una clase interfaz “IValidatorCallback” que se encarga de gestionar las terminaciones de la validación por fallos o por terminación de la ejecución. Ya que es la clase “GeneratorComponent” quien se encarga de llamar a la herramienta de generación, es esta quien hereda de la interfaz y quien es llamada por la clase “BehaviorList” cuando falla la validación, pasándole una lista de todas las acciones ejecutadas hasta el momento y la acción en la que ha fallado. Es entonces cuando la clase “GeneratorComponent” interrumpe el modo de juego y vuelve a llamar a la herramienta con el pseudocódigo fallido, el contexto dado por “BehaviorList” y el prompt con la acción original y vuelve a intentar la ejecución cuando la herramienta arregla el pseudocódigo.

Para validar los objetivos específicos de los comportamientos se ha creado una clase interfaz “ObjectiveValidatorBase” que contiene un método para comprobar si el objetivo se ha cumplido y un mensaje de error indicando que el objetivo ha fallado. De esta interfaz deben heredar componentes de Actor para que los objetos “BTGeneratorManager” los puedan tener y puedan hacer la comprobación de los objetivos una vez se acabe la ejecución. Si la comprobación del objetivo es fallida se le pasa a la herramienta el mensaje de error y la lista de tareas ejecutadas, así como el pseudocódigo fallido y el prompt original y se vuelve a intentar con un pseudocódigo arreglado.

Una vez se han tenido los métodos de validación de los [Behavior Trees](#) se han creado tres entornos con diferentes pruebas. El primero de ellos consta de tres flores de distintos colores: rojo, azul y amarillo. Estas flores han sido creadas mediante Blueprints de Unreal, y cuando se inicia el modo de juego se guardan en una lista de una instancia propia de “Game Instance”, la cual es una clase que persiste durante todo el juego, independientemente de si cambias de mapa. En este entorno se le pide a la herramienta que genere un [Behavior Tree](#) donde recoja una única flor de cada tipo, independientemente del orden, con el siguiente prompt: “NPC that has to choose and pick one flower red, one flower yellow, and one flower blue. In order to pick a flower it has to be first in the position of the flower.”. Al ser una tarea sencilla la herramienta solo ha necesitado un intento y un tiempo de ejecución de 84.84 segundos para crear un [Behavior Tree](#) válido y que cumpla con el objetivo.

El segundo de los entornos consta de tres flores azules, una flor roja y una flor amarilla, y aquí se le pide a la herramienta que genere un [Behavior Tree](#) que recoja tres flores azules y después una roja mediante el siguiente prompt: “NPC that has to choose and then pick first three blue flowers, and, after that, has to choose and pick one red flower. In order to pick a flower it has to be first in the position of the flower”. Al igual que en el caso anterior, solo ha necesitado de un intento y un tiempo de 45.46 segundos para realizar un [Behavior Tree](#) válido. En la figura 3.7 se puede ver una captura de pantalla de este entorno.

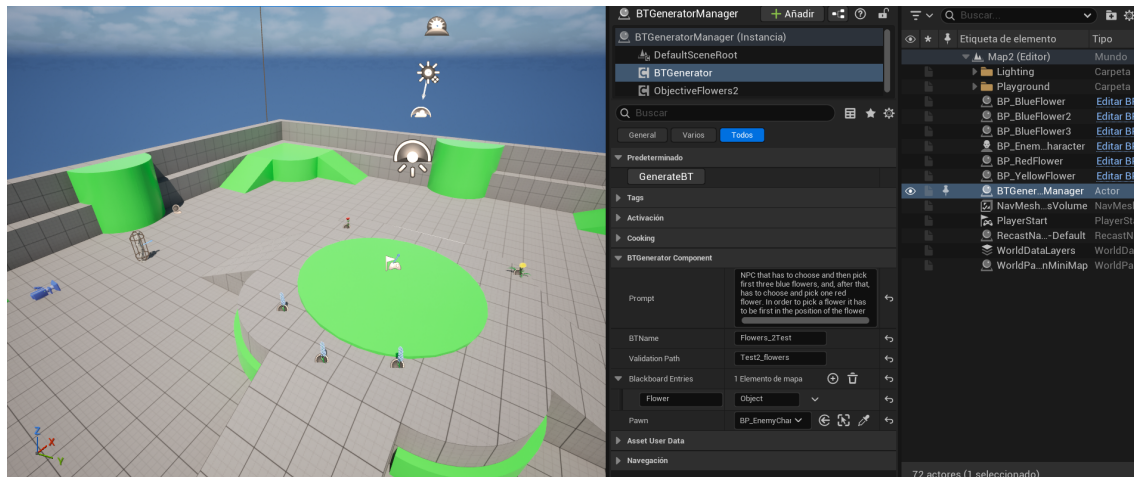


Figura 3.7: Captura de pantalla del segundo entorno de pruebas de validación.

El tercer entorno consta de un enemigo y un punto de patrulla (otro Blueprint de Unreal que, de forma similar a las flores, se añade a la instancia de “GameInstance” al iniciar la partida). Aquí se le pide a la herramienta que genere un comportamiento clásico en videojuegos: un NPC que patrulle el escenario por distintos puntos y, que si ve al jugador, le persiga para atacarle. Para ello se ha usado el siguiente prompt: “NPC that has to do one of this two options: - If the objective is at his line of sight it has to: 1. chase him 2. move to his position 3. if is near the objective, it has to attack him. - Otherwise, it has to: 1. select a waypoint 2. move to the waypoint”. A pesar de ser una petición más compleja debido a la naturaleza del comportamiento y al aumento de número de variables de **Blackboard** (cuatro variables, dos condicionales para ver si el objetivo está en el punto de mira y si el objetivo está cerca, el punto de patrulla a elegir y el objetivo a perseguir), la herramienta ha sido capaz de generar en 153.1 segundos un **Behavior Tree** válido a la primera en cuanto a cumplimiento de objetivos y respetando las dependencias, aunque sí se ha tenido que modificar alguna tarea que requería de una variable de la **Blackboard** que el segundo LLM no le ha adjudicado ninguna variable. En la figura 3.8 se puede ver una captura de pantalla de este tercer entorno.

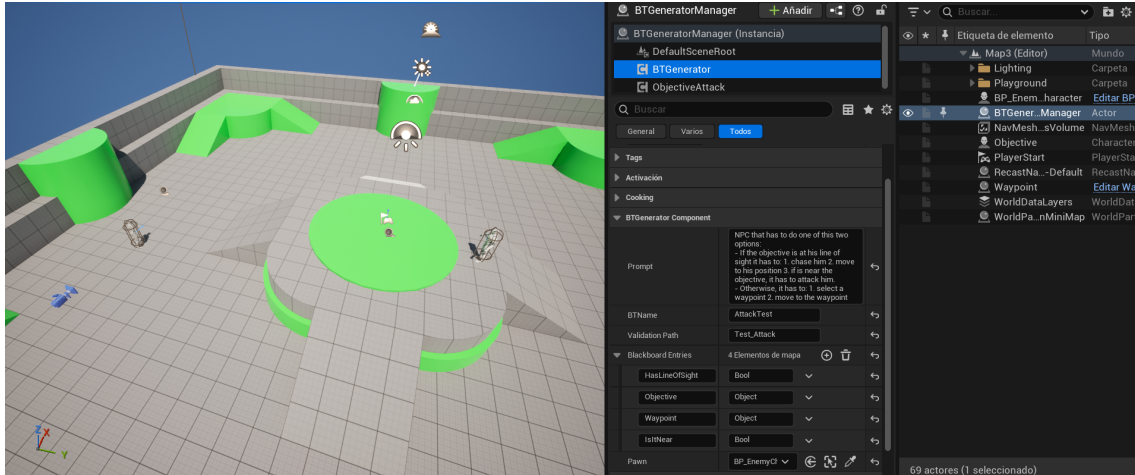


Figura 3.8: Captura de pantalla del tercer entorno de pruebas de validación.

Con los resultados de esta validación, resumidos en la tabla 3.2, se ha podido establecer un baseline del funcionamiento de la herramienta para, finalmente, realizar pruebas con usuarios.

Tabla 3.2: Resultados de las pruebas de validación

Entorno	Intentos	Tiempo de ejecución
1	1	84.84 segundos
2	1	45.46 segundos
3	1	153.1 segundos

3.5. Pruebas con usuarios

Como bien se ha mencionado en la sección 1.2, lo que se busca en este trabajo es una herramienta capaz de generar de manera similar a los humanos un **Behavior Tree** funcional o, al menos, una aproximación que sea capaz de disminuir el trabajo a los diseñadores de videojuegos. Con este objetivo en mente se ha diseñado unas pruebas con usuarios para comprobar si se cumplen los objetivos planteados.

Para ello se ha realizado un plan de pruebas donde se evalúan los tiempos de ejecución de generación y ejecución de los **Behavior Tree**, la comparación entre los tiempos de un **Behavior Tree** generado por un humano y uno generado por la herramienta, los intentos de generación de un **Behavior Tree** y la calidad de la herramienta en cuanto a usabilidad y funcionalidad. Para estas métricas las hipótesis iniciales eran las siguientes:

- H1: La herramienta es capaz de generar **Behavior Trees** más rápido que un diseñador que los hace a mano
- H2: Los **Behavior Trees** generados realizan una ejecución similar a los realizados por un humano

- H3: La calidad de los [Behavior Trees](#) generados por la herramienta es un poco menor que los generados por un humano, pero les permite a los diseñadores tener un punto de partida más rápido que puede ser fácilmente modificable, disminuyendo así el tiempo de trabajo
- H4: La herramienta de generación de [Behavior Trees](#) es intuitiva y fácilmente de utilizar.

Para poder comprobar estas hipótesis se han buscado usuarios que cumplan con el perfil de un diseñador de videojuegos que haya trabajado con Unreal Engine y sus [Behavior Trees](#). El procedimiento con los usuarios ha sido el siguiente:

1. Con el fin de mantener el anonimato, se le asigna un identificador único a cada usuario.
2. Se le pide que haga un [Behavior Tree](#) de un [NPC](#) que patrulla hasta que ve a un objetivo, que debe perseguirlo hasta estar lo suficientemente cerca como para atacarlo.
3. Se le pide que haga un [Behavior Tree](#) de un [NPC](#) que va recogiendo flores hasta que ve a un objetivo, que en ese caso debe huir a un punto del mapa.
4. Se les cronometra desde el momento en el que se les indique que pueden empezar hasta que consigan una ejecución que satisfaga a los objetivos propuestos.
5. Se les pide que realicen los mismos [Behavior Tree](#) con la herramienta.
6. Se les pide que rellenen un formulario sobre la usabilidad y calidad de la herramienta.

Para cada [Behavior Tree](#) que debería ser creado se hicieron dos escenarios distintos, uno en el que se deba usar la herramienta y otro en la que el [NPC](#) ejecuta un [Behavior Tree](#) creado a mano. Para el primer [Behavior Tree](#) se ha hecho una copia del tercer escenario mencionado en la sección 3.4, con un objetivo y un punto de patrulla cerca de él en el mapa. El segundo escenario consta de un objetivo, dos flores rojas (una de ellas en frente el objetivo para que el [NPC](#) tenga que pasar por delante de él) y un punto de patrulla alejado del objetivo.

El formulario, que se puede encontrar en el apéndice D, mide la usabilidad y la calidad de la herramienta frente a [Behavior Trees](#) creados por un humano. Para la usabilidad se ha decidido evaluar mediante el System Usability Scale (SUS), introducido en [Brooke \(1996\)](#), que consiste en un cuestionario estandarizado de 10 ítems que siguen la escala de Likert de 5 puntos, que van desde “Totalmente en desacuerdo” a “Totalmente de acuerdo”. Este método de evaluación se ha decidido seguir por evidencias de su robustez en estudios como [Bangor et al. \(2008\)](#) o [Peres et al. \(2013\)](#).

Para la calidad de la herramienta se les han realizado preguntas sobre los resultados esperados de tiempos de ejecución, diferencias entre la calidad y similitud de sus [Behavior Trees](#) y los generados por la herramienta, y su funcionalidad como punto de partida. Finalmente se ha habilitado un campo de texto no obligatorio de

respuestas libres donde los usuarios podrían expresar su opinión sobre los [Behavior Trees](#) generados y la herramienta.

Debido al perfil específico que se necesitaban para las pruebas solo se han podido realizar pruebas con 3 usuarios, por lo que las conclusiones sirven como una aproximación pero no se deben tomar como concluyentes.

En cuanto a la usabilidad podemos ver en la figura 3.9 que se ha obtenido una media de 90.83 sobre 100, lo cual supera con creces la media de 68 establecida por el artículo como [Hyzy et al. \(2022\)](#), por lo que dadas estas pruebas se puede decir que la herramienta cumple con el objetivo de accesibilidad, haciéndola así fácil de utilizar y confirmando con los datos dados H4.

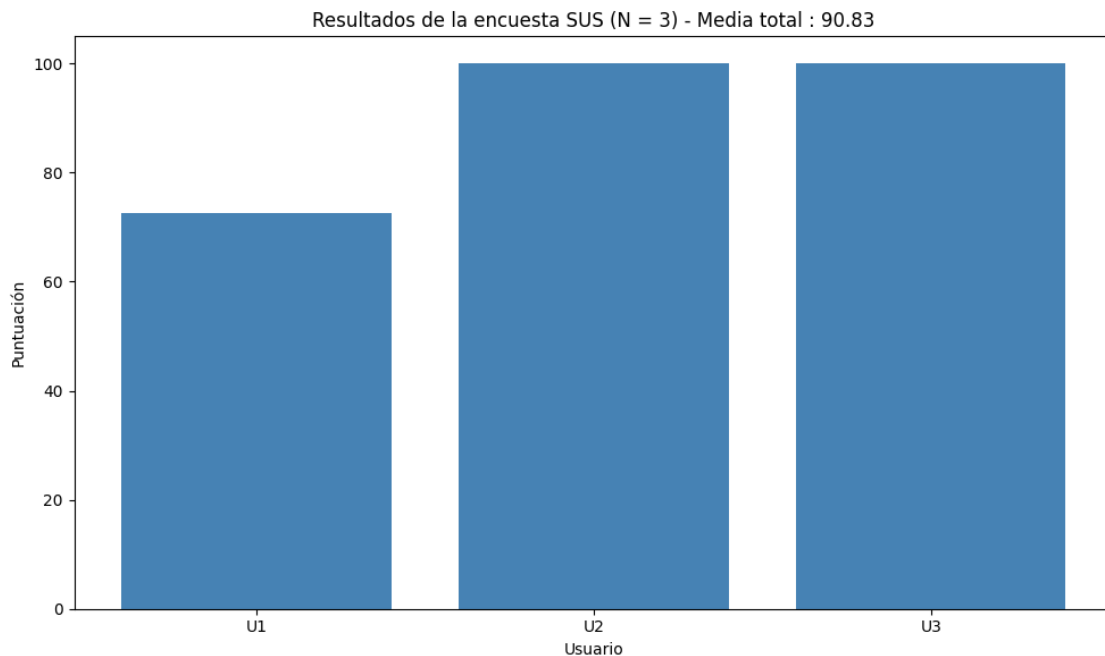


Figura 3.9: Resultados de la encuesta SUS de los usuarios sobre la usabilidad de la herramienta.

La comparativa de los tiempos podemos verla en la figura 3.10, en donde a excepción del primer [Behavior Tree](#) que ha tardado más en generarlo la herramienta que el segundo usuario, hay una diferencia significativa en cuanto a la tardanza, beneficiando a los generados por la herramienta. Además, como podemos ver en la figura 3.11, los usuarios tienen una opinión favorable sobre los tiempos de ejecución de la herramienta con una media de 4.33/5, afirmando así la hipótesis inicial de generar de forma más rápida los [Behavior Trees](#) con unos tiempos de generación adecuados (H1).

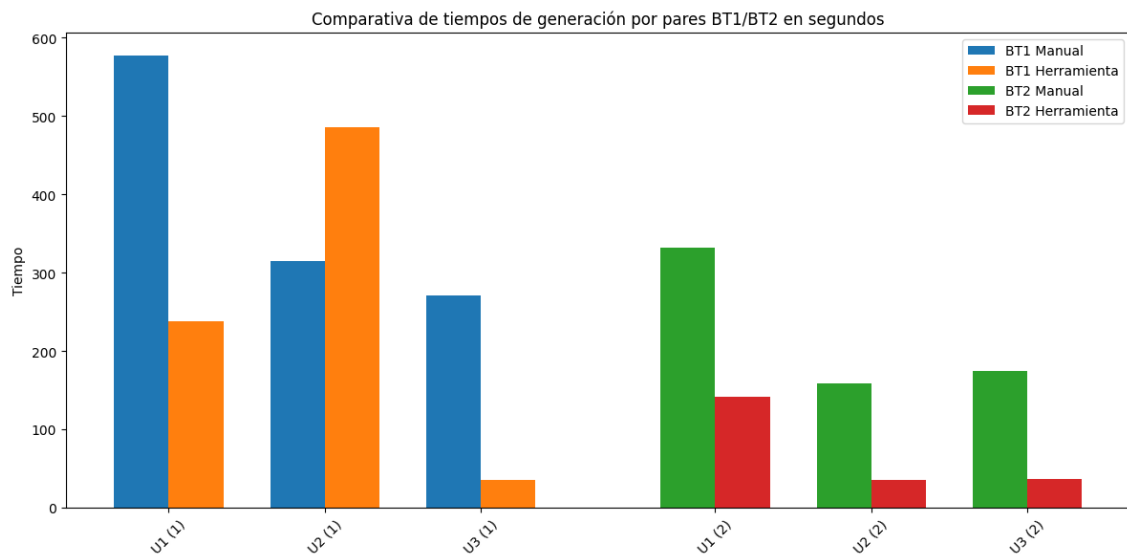


Figura 3.10: Comparación de cuánto han tardado en crear los usuarios los [Behavior Trees](#) junto a cuánto ha tardado la herramienta.

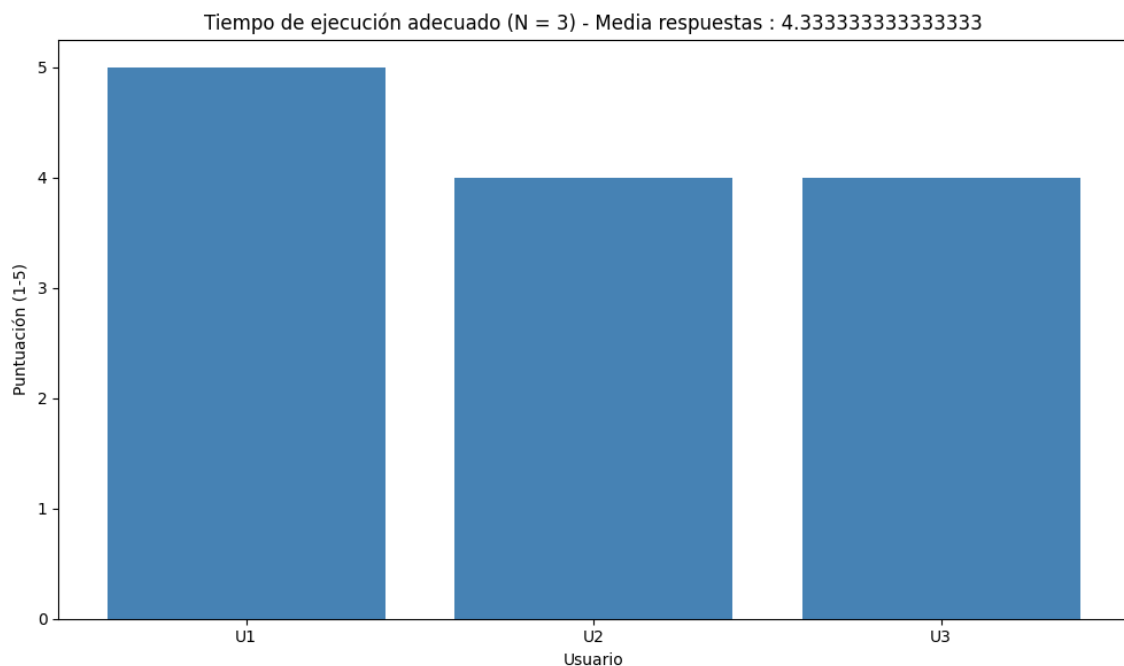


Figura 3.11: Opinión de los usuarios sobre los tiempos de ejecución de la herramienta del 1 al 5, siendo 1 una opinión desfavorable y 5 la más favorable.

Siguiendo con la calidad de los [Behavior Trees](#) podemos observar en las figuras 3.12 y 3.13 que la calidad de los [Behavior Trees](#) es inferior a los generados por un humano (con una media en las respuestas de 4.0/5), pero que cumplen con la ejecución esperada (con una media de las respuestas de 4.0/5 también). Esto se puede explicar gracias a las respuestas de las preguntas sobre si los [Behavior Trees](#) generados tenían más nodos de los necesarios y si eran distintos a los generados por

los humanos, en los que todos los usuarios respondieron que la herramienta añadía algunos nodos extra (tanto compuestos como tareas innecesarias) y que esto hacía que los [Behavior Trees](#) generados fuesen distintos a los creados manualmente, dando una calidad inferior pero con una ejecución similar, confirmando así la hipótesis H2.

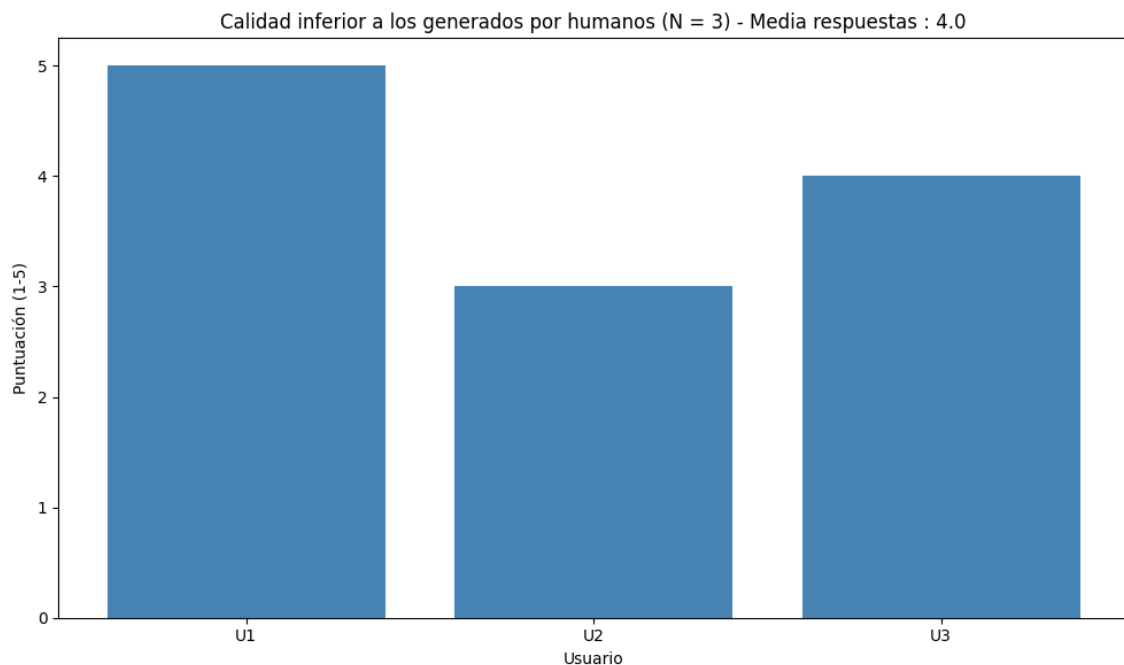


Figura 3.12: Respuestas de los usuarios sobre la calidad de los [Behavior Trees](#) generados por la herramienta, donde 1 es que no están de acuerdo con la afirmación de que la calidad de los [Behavior Trees](#) generados por la herramienta son inferiores y 5 es que están completamente de acuerdo.

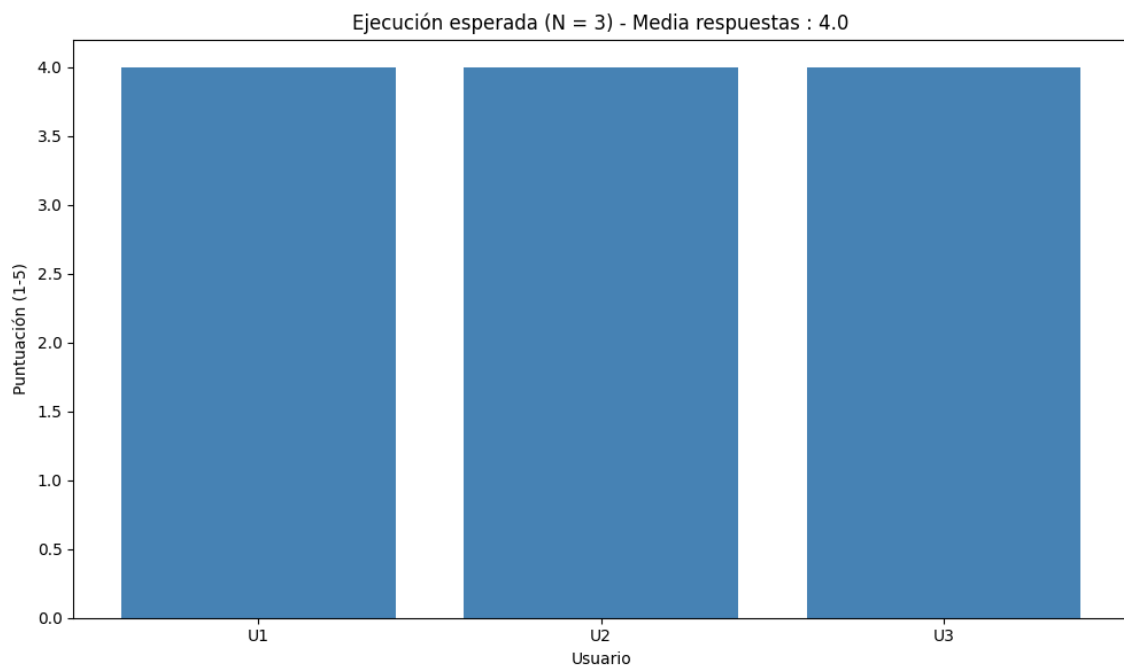


Figura 3.13: Respuestas de los usuarios sobre la ejecución esperada de los [Behavior Trees](#), donde 1 es que consideran una mala ejecución y 5 significa una ejecución correcta.

Finalmente y junto a los resultados del apartado de calidad, la comprobación de H3 se puede apoyar en los resultados de las figuras 3.14, 3.15 y 3.16, donde podemos ver que los usuarios han tenido una buena opinión sobre la idea de la usabilidad de la herramienta como apoyo para los diseñadores, respondiendo con un 4.67/5 a la opinión de que sirve como buen punto de partida y que, gracias a la facilidad de modificación de los JSONs generados (con una media de respuestas de 4.33/5), la herramienta puede servir para disminuir la carga de trabajo de los diseñadores (con una opinión de 4.67/5).

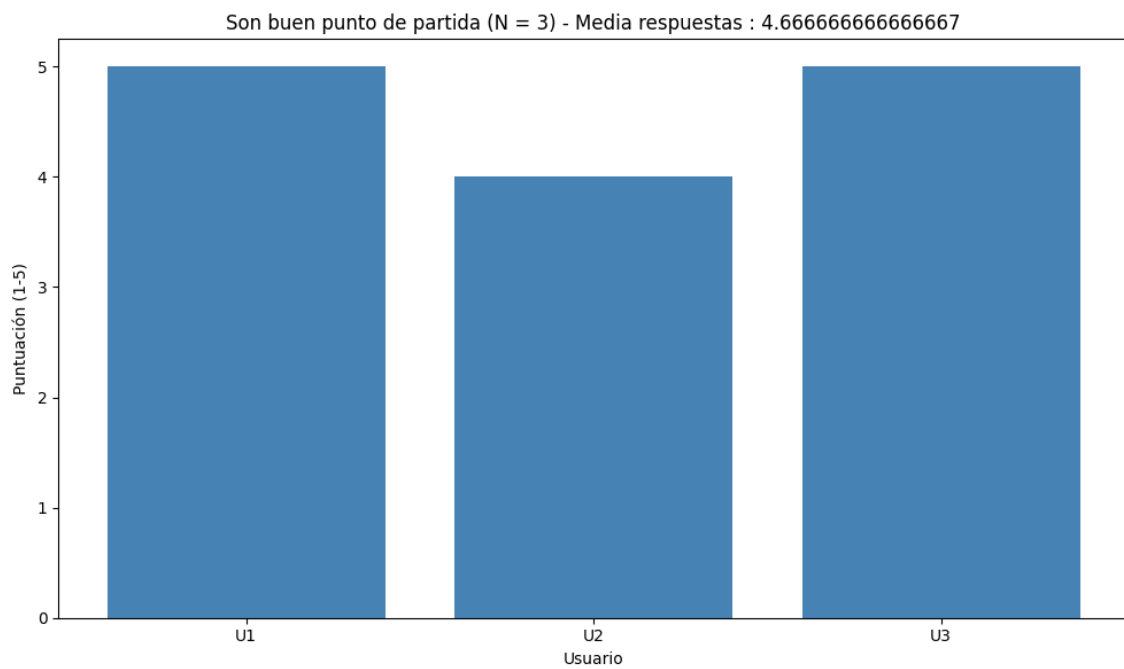


Figura 3.14: Respuestas de los usuarios donde indican si los [Behavior Trees](#) generados sirven como punto de partida (5) o no (1)

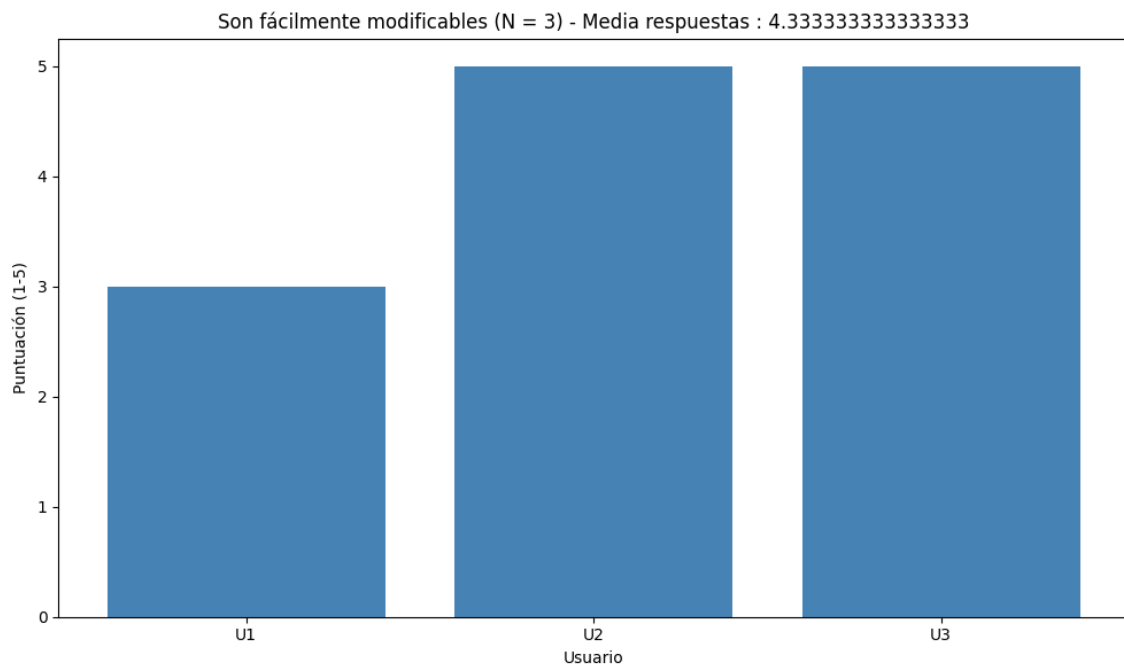


Figura 3.15: Respuestas de los usuarios sobre la facilidad para modificar los [Behavior Trees](#) generados, donde 1 es significa que es difícil y 5 significa que es fácil.

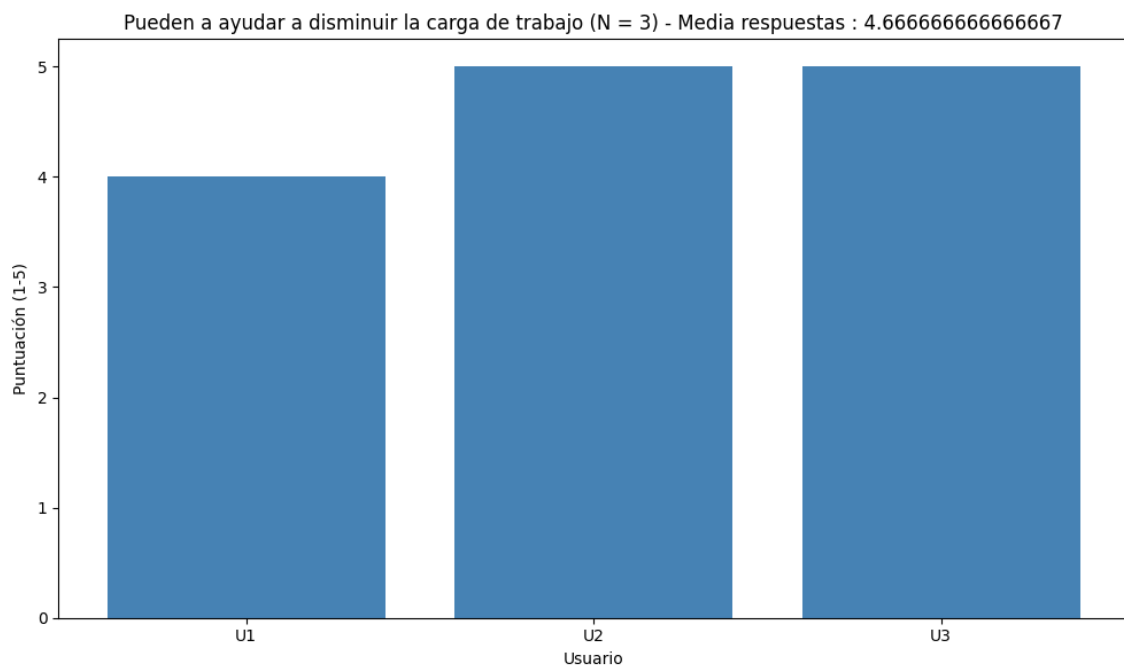


Figura 3.16: Respuestas de los usuarios sobre su opinión de si la herramienta puede disminuir (5) o no (1) la carga de trabajo de los diseñadores.

Conclusiones y Trabajo Futuro

4.1. Conclusiones

El objetivo de este estudio consistía en la creación de una herramienta que, dadas unas tareas y condicionales, fuese capaz de generar [Behavior Trees](#) que sirviese como base para diseñadores de videojuegos, aliviando así la carga de trabajo de éstos teniendo una base sobre la que trabajar. Para ello se escogió el motor Unreal Engine 5, donde se crearon varios escenarios así como una serie de tareas y condicionales para poder probar las generaciones de los [Behavior Trees](#). Con la base creada se continuó con el modelo de la herramienta, en la que se decidió el flujo de ejecución y generación. Se procedió con la implementación del generador de [Behavior Trees](#), en la que se usaron dos [LLMs](#) para las tareas de generación de pseudocódigo y selección de variables de [Blackboard](#) para las tareas y decoradores que lo necesitasen. Esta implementación consistió en el uso de técnicas de *prompt engineering* con el fin de disminuir las alucinaciones de los modelos. A su vez se implementó un sistema de [RAG](#) para poder dotar al modelo de ejemplos útiles. Con este fin se buscó un dataset lo suficientemente extenso como para que el sistema fuese útil, por lo que se decidió usar el dataset publicado en [Lykov y Tsetserukou \(2023\)](#). Este dataset ha sido limpiado y traducido al pseudocódigo usado en la generación. Una vez se tenía la generación implementada se hizo un sistema de validación de los [Behavior Trees](#) para verificar que cumplen los objetivos propuestos y la dependencia de las acciones. Para las dependencias se implementó un NFA, y para los objetivos una clase auxiliar que comprobase las condiciones necesarias al finalizar la ejecución. Junto a esto se implementó un sistema en el que, si falla alguna de las validaciones, se le pide a la herramienta que arregle el pseudocódigo dado el fallo que haya tenido el anterior. Con el fin de validar experimentalmente los resultados se diseñaron unas pruebas de usuarios en los que se comprobaba la usabilidad y la funcionalidad de la herramienta.

Las conclusiones a las que se han llegado de diferentes partes del trabajo se detallan en las siguientes secciones.

4.1.1. Conclusiones de los escenarios implementados

Debido al fin del trabajo de desarrollar una herramienta que sirva como apoyo para diseñadores de videojuegos, se implementaron diferentes escenarios en un proyecto de Unreal Engine 5. Los escenarios, tareas y condicionales implementados han cumplido su función como marco en el que probar la generación de los [Behavior Trees](#), consiguiendo un marco en los que los diseñadores de videojuegos se sientan acostumbrados y se consiga la opacidad buscada de la herramienta frente a estos.

4.1.2. Conclusiones de la herramienta generadora de [Behavior Trees](#)

Tras los intentos mencionados en la sección 3.3, se ha concluido que la forma de generar [Behavior Trees](#) que mejores resultados ha dado ha consistido en convertir las peticiones y las tareas y decoradores disponibles en un formato de pseudocódigo que luego ha podido ser traducido y completado con el uso de variables de la [Blackboard](#) gracias a un segundo modelo [LLM](#).

Para la generación de pseudocódigo se ha necesitado del modelo “mistral-large-3”, con un total de 675B de parámetros. La ventaja de esto es que la generación resulta bastante correcta, tanto en la calidad de las generaciones como en el formato resultante, ya que modelos más pequeños no conseguían generaciones en las que se cumpliesen los objetivos o las dependencias y el formato del pseudocódigo pedido solía no cumplir con la sintaxis especificada. Otra ventaja de usar un modelo grande mediante [API](#) es el poco tiempo que consigue generar el pseudocódigo, a diferencia de otros modelos en local que tendrían una latencia mayor. Sin embargo, la dependencia de [APIs](#) externas puede traer problemas a largo plazo, ya que pueden actualizar los términos de uso o el listado de precios o directamente cerrar el modelo. Es por esto último que se estima necesario el estudio del uso de modelos locales mediante técnicas de [Fine-tuning](#) para verificar un acercamiento de los resultados observados sin las desventajas mencionadas siempre y cuando el tiempo de generación no aumente de forma significativa.

Para la asignación de variables de [Blackboard](#) se ha utilizado el modelo local “llama3:8b”, que cuenta con 8B de parámetros. Esto se decidió así debido a las peticiones limitadas a la [API](#) de Mistral. El hecho de ser un modelo más pequeño que el anterior ha resultado en algunos fallos a la hora de fijar las variables, siendo un motivo por el que los usuarios tengan que revisar y cambiar los JSONs generados. Sin embargo estos fallos no suelen ser habituales, por lo que dadas las pruebas de validación se puede concluir que cumple con su cometido. A pesar de no ser habituales estos fallos se considera igualmente la investigación del uso de otro modelo más grande y la búsqueda de datos para poder realizar y comparar un [Fine-tuning](#) con el uso de un sistema [RAG](#) para este modelo con el fin de reducir las alucinaciones y aumentar la precisión, lo que implicaría disminuir la intervención humana y reducir aún más la carga de trabajo de los diseñadores.

Para el sistema [RAG](#) montado se buscaron datasets de [Behavior Trees](#) específicos de videojuegos, pero la ausencia de este tipo de datos ha implicado que se haya usado un dataset de [Behavior Trees](#) de robótica para posteriormente traducirlos al

formato buscado para el trabajo. A pesar de ser un ámbito distinto estos ejemplos han funcionado para la generación de pseudocódigo, ya que las tareas de robótica incluidas en el dataset se pueden asemejar a las tareas que debe realizar un NPC, consiguiendo unos [Behavior Trees](#) con mayor precisión (menor número de nodos adicionales o selección de nodos de composición más certera). Un aspecto negativo de este sistema es que el tiempo de generación aumentaba en las ocasiones en las que no estaba guardado en memoria, retrasando la generación unos cuantos segundos. Para disminuir las alucinaciones también se ha hecho uso de varias técnicas de *prompt engineering*, tales como “Few-shot prompting” o “Chain-of-Thought”, consiguiendo así [Behavior Trees](#) que logran cumplir con los objetivos propuestos para los comportamientos pedidos. Una técnica fundamental ha sido “Self-Critique Prompting”, ya que en varios casos ha logrado corregir errores de alucinaciones en cuanto a que el modelo se inventaba tareas, decoradores o la sintaxis del pseudocódigo.

Con todo esto se puede concluir que, aún no conseguir resultados completamente perfectos, la herramienta es capaz de generar [Behavior Trees](#) que sirvan como una buena base para generar comportamientos en los que los diseñadores solo tengan que cambiar unos pocos detalles para lograr el objetivo final, reduciendo así su carga de trabajo.

4.1.3. Conclusiones de la validación automática

Para poder validar y corregir cuando es debido de forma automática los [Behavior Trees](#) generados se han propuesto dos tipos de validaciones. Estas validaciones no son obligatorias de usar en la herramienta, ya que pueden suponer un aumento de carga de trabajo para los diseñadores que no compensa el trabajo ahorrado de generar los [Behavior Trees](#) a mano. Sin embargo esta herramienta puede ayudar a la automatización del proceso con el arreglo de fallos de forma automática.

La primera validación realizada consiste en la verificación del cumplimiento de dependencias de las tareas. Debido a la probable duplicación de tareas en diferentes puntos del [Behavior Tree](#) se ha optado por la implementación de un NFA. Debido a que no se ha encontrado la forma de poder verificar en cada tick qué nodos está ejecutando un [Behavior Tree](#) de Unreal Engine, se han tenido que hacer de forma explícita las llamadas al NFA desde las tareas para indicar cuál se está ejecutando. A pesar de este aspecto negativo, el NFA ha resultado cumplir su función de forma correcta, consiguiendo que la herramienta corrija los fallos de dependencias en el pseudocódigo con la información que se le transmite (pseudocódigo fallido, acción en la que ha fallado y una lista con todas las acciones ejecutadas previamente).

La segunda validación consiste en un sistema para validar los objetivos de los comportamientos buscados. El sistema consiste en un componente de actor al que se llama al finalizar la ejecución, el cual si falla la verificación devuelve un mensaje de error y llama de nuevo a la herramienta. Este método ha conseguido que, en los casos en los que los [Behavior Trees](#) resultaban fallidos, la herramienta sea capaz de comprender mejor la acción buscada y reestructurar el [Behavior Tree](#) para conseguir uno correcto.

Para esta validación se probaron tres escenarios en los que se probaba una generación sencilla sin apenas dependencias, otra sencilla en la que había más dependencias

de tareas y una última con varias variables de [Blackboard](#) y un comportamiento más complejo. Para estos tres escenarios se consiguieron unas generaciones válidas tanto en objetivos como en dependencias.

4.1.4. Conclusiones de las pruebas de usuarios

Como bien se ha comentado en la sección 3.5 estas pruebas se han realizado con un número reducido de usuarios ($N=3$), por lo que las conclusiones no se deben tomar como definitivas sino como una aproximación.

Teniendo en cuenta el objetivo final de este trabajo se formularon cuatro hipótesis para las pruebas de usuario:

1. H1: La herramienta es más rápida que un humano
2. H2: La generación de los [Behavior Trees](#) generados es similar a los realizados a mano
3. H3: La calidad de los [Behavior Trees](#) generados es ligeramente menor pero sirve como punto de partida.
4. H4: La herramienta es intuitiva y fácil de usar.

Tras las pruebas de usuario y una siguiente encuesta se han llegado a las siguientes conclusiones sobre cada hipótesis:

1. H1: Según la observación experimental los usuarios tardaban más en realizar los [Behavior Trees](#) a mano que en generarlos con la herramienta.
2. H2: La herramienta generaba [Behavior Trees](#) que conseguían una ejecución similar a los realizados a mano
3. H3: Los [Behavior Trees](#) generados colocaban nodos extra (tanto nodos de composición como tareas) o no lograban asignar correctamente todas las variables de [Blackboard](#) a las tareas, haciendo que los [Behavior Trees](#) generados fuesen de una calidad inferior a los humanos, aunque formaba una base para que la intervención humana sea solo necesaria en cambiar esos aspectos.
4. H4: La encuesta SUS realizada para medir la usabilidad dio un resultado de 90.83/100, por lo que se confirma la facilidad de uso de la herramienta para diseñadores.

Con estas pruebas se puede ver que se cumplen las cuatro hipótesis pensadas inicialmente, aunque se necesitan más usuarios que pidan el perfil pedido (diseñadores de videojuegos que haya trabajado con Unreal Engine y su sistema de [Behavior Trees](#)) para poder cerrar las conclusiones de forma definitiva.

4.1.5. Limitaciones

Este trabajo ha sufrido una serie de limitaciones en cuanto a limitaciones técnicas y a la escasez de usuarios disponibles para las pruebas.

Las limitaciones técnicas se han podido comprobar a la hora de la selección de modelos para generar los [Behavior Trees](#). Como bien se ha mencionado anteriormente los dos modelos, a pesar de cumplir de forma correcta su objetivo, presentan una serie de limitaciones debido a las capacidades computacionales disponibles. El modelo generador de pseudocódigo es un [LLM](#) que no funciona de forma local, teniendo que depender de una [API](#) de terceros. El modelo que asigna las variables de [Blackboard](#) es un modelo bastante pequeño que puede cometer errores. Además, el sistema [RAG](#) se ha construido con [Behavior Trees](#) de robótica y no de videojuegos y que, adicionalmente, puede tardar varios segundos en construirse. Otra limitación es la falta de automatización en ciertos aspectos, como en la suscripción de las tareas a la [Abstract Factory](#) así como la asignación de su descripción. Aunque los usuarios encuestados encontrasen sencilla la modificación de los [Behavior Trees](#) generados, es posible que en el futuro busquen cambios adicionales en el comportamiento o necesiten reestructurarlo de forma sencilla, lo cual en el estado actual supondría pedirle otra generación distinta a la herramienta en vez de partir con el [Behavior Tree](#) ya existente.

Finalmente la limitación más grande ha sido la falta de usuarios para poder realizar las pruebas de usuarios, haciendo imposible concluir de forma definitiva las hipótesis planteadas siguiendo los objetivos.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- Con capacidades computacionales mayores estudiar el uso de modelos locales de mayor tamaño que 8B de parámetros para los modelos usados.
- Conseguir una mayor colecta de datos de [Behavior Trees](#) de videojuegos para comprobar si uso de [Fine-tuning](#) en modelos locales alcanzaría o mejoraría los resultados actuales.
- Realizar un sistema [RAG](#) que sea capaz de ingerir los [Behavior Trees](#) correctos generados por la herramienta.
- Mejorar el sistema [RAG](#) para que se guarden en disco los [Embeddings](#) y se puedan buscar por metadatos gracias al uso de Qdrant.
- Implementar un agente [LLM](#) y una interfaz visual para cambiar los [Behavior Trees](#) generados de forma más sencilla y se le pueda dar feedback al generador mediante uso de lenguaje natural.

- Implementar un agente [LLM](#) que pueda leer las acciones y descripciones de los proyectos para automatizar su suscripción en la [Abstract Factory](#) y su paso a la herramienta.
- Recopilar más usuarios para las pruebas con el fin de conseguir un número estadísticamente significativo para concluir definitivamente las hipótesis propuestas.

Introduction

In recent years, the advancement of immersive technologies such as [VR](#) has brought about a significant change in how users interact in digital environments. Major companies like Meta and Apple have invested in the development of hardware for these technologies, launching devices like the Meta Quest 3 and Apple Vision Pro, and creating social interaction platforms in the [metaverse](#) with the development of Meta Horizon Worlds. This technology has opened new opportunities to explore more natural modes of communication in virtual spaces through non-verbal communication.

Non-verbal communication refers to the transmission of messages without the use of words, relying instead on images and gestures. In our daily lives, it plays a crucial role in human interactions, yet its representation in virtual environments is often limited and dependent on predefined actions such as animations or "emotes" already integrated into the game. Therefore, the development of tools that can capture and interpret these gestures in real-time represents a significant step towards creating more expressive and realistic virtual environments.

4.3. Motivation

As mentioned in the introduction, the use of non-verbal communication in virtual environments is limited and predefined by their design, restricting a fundamental part of everyday human expression. Due to this, and as seen in articles like [Neverova et al. \(2013\)](#) or [Herbert et al. \(2024\)](#), gesture detection is a field of study currently being explored to address this issue. For these reasons, it is interesting to investigate ways to expand this representation of non-verbal communication so that interaction between users and [NPCs](#) becomes more natural.

This work is therefore part of the research line focused on developing tools that can recognize gestures performed by a user using a motion capture suit, with the aim of creating more immersive and natural virtual environments for users. On the other hand, a large amount of data is required to create [AI](#) models that enable the development of the mentioned tools. In the case of this project, the necessary data consists of a sequence of 3D coordinates representing the positions and rotations of the skeleton's bones throughout the animations, resulting in relatively large data sizes. This necessitates the creation of complex models capable of processing a

significant amount of input data. However, as we will discuss throughout the work, it has not been possible to find datasets containing this type of data. As a result, a significant contribution of this work to the field has been the creation of a dataset through the extraction of animations from a specialized bank and the motion capture of users.

Given these reasons, it can be concluded that the lack of natural methods to leverage non-verbal communication in virtual environments and the scarcity of animation datasets have motivated the objectives mentioned, which we will expand upon below.

4.4. Objectives

The general objective of this project is to implement an [AI](#) model that, with low latency, allows real-time identification of gestures performed with a motion capture suit, aiming to enhance non-verbal communication in virtual environments. To achieve this objective, the following goals have been proposed during the development of the work:

1. Search for animation datasets that are considered relevant for communicating with an [NPC](#) in a video game. These animations include dancing, greeting, pointing, sitting, fighting, and running.
2. Automatic extraction of the aforementioned animations from animation banks and subsequent processing.
3. Extraction of animations through motion capture from a heterogeneous group of users.
4. Implementation, training, and comparison of different [AI](#) models, including neural networks (LSTM, CNN, and RNN) as well as classical classification models (Random Forest).
5. Development of a final application as a demonstration to showcase the results in [VR](#) to enhance immersion.

As a significant secondary objective, due to the lack of animation datasets, it has been proposed to publish the resulting datasets, both from artificial animations and user captures.

4.5. Work Plan

The work plan consists of several steps:

1. Search for a dataset: generate a sufficiently large dataset with multiple examples of gestures to adequately train different models. This dataset should primarily consist of data from real users to identify the most natural gestures possible.

- 2. Implementation of AI models: implement various AI models to compare them and determine which one is most suitable based on prediction speed and accuracy. The implementation of these models should include: training, a way to extract final model statistics, hyperparameter search, confusion matrix generator, and compatibility with a common user interface for all trainings.
- 3. Development of a user interface for training and monitoring different models: develop a web interface so that users with less programming and AI experience can train models capable of predicting the gestures they need without modifying the code or needing to understand the internal workings of the codebase.
- 4. Development of a final application: create an application for Oculus Quest as a demo that connects to the chosen model and allows real-time usage demonstration.

The development of tasks and their planning can be seen in Figure 4.1.

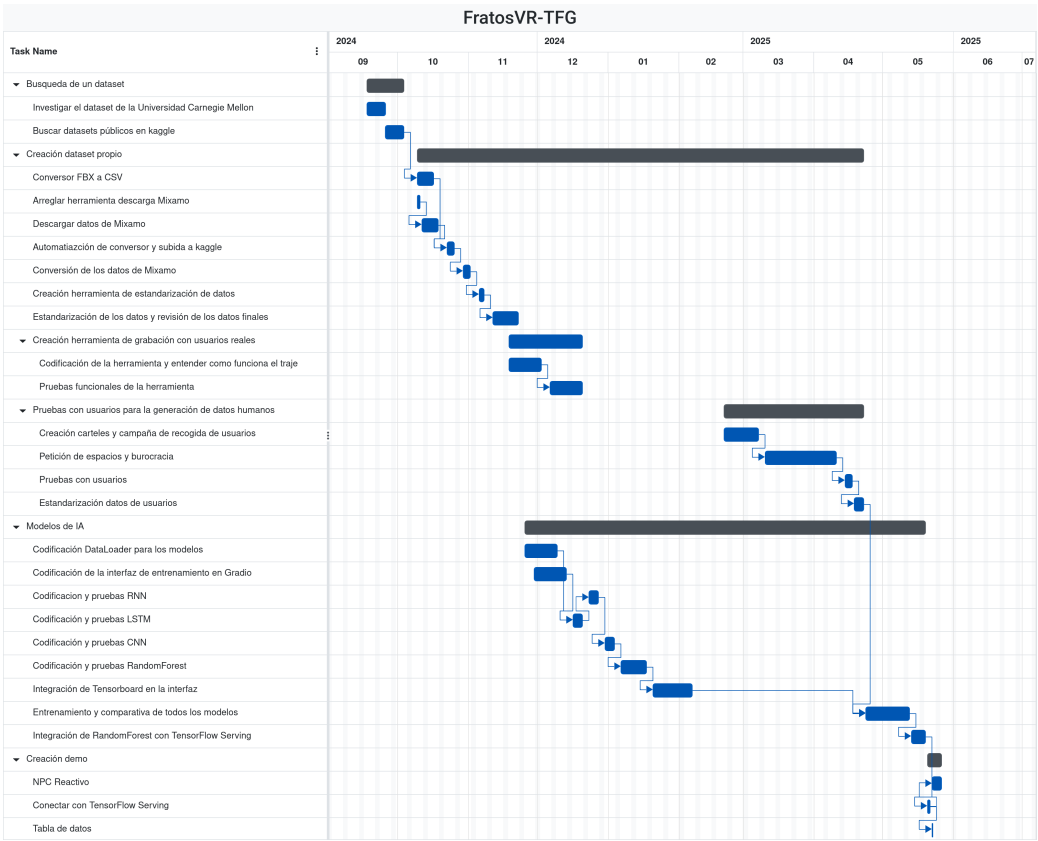


Figure 4.1: Gantt chart of the project

Conclusions and Future Work

4.6. Conclusions

The objective of this study was to create an [AI](#) model capable of detecting a series of gestures from users wearing a motion capture suit. To achieve this, datasets of gestures that matched the study's constraints were sought. Since no easily adaptable datasets with the required gestures were found, an animation dataset was created, consisting of both computer-generated data and data collected from real users. The computer-generated animations were converted to [CSV](#) format using a tool created by the authors of the study, while the animations collected from real users were processed with another tool developed by the same authors. After training various [AI](#) models, it was concluded that the [Random Forest](#) model is the best choice for several reasons. Following this, a demonstration application was created to showcase the final results.

The conclusions of each part of the work are presented in the following sections in more detail.

4.6.1. Dataset Search Conclusions

After the search for animation datasets, it was concluded that there are not enough public datasets available for the objective of this study.

These conclusions led us to the need to create our own dataset by collecting animations from animation banks and through user trials.

4.6.2. Conclusions of the Animation Extraction Tool

The gesture conversion tool fully meets its objective, as it only requires specifying the name of the folder containing the downloaded gestures. It automatically processes the gestures in Unity and uploads them to Kaggle. Additionally, the tool is independent of the number of gestures to be converted and the number of animations available for those gestures, achieving great scalability. This allows for the introduction of new gestures without requiring any changes to the tool.

4.6.3. Conclusions of the User Data Collection

As can be seen in the figures in Appendix ??, the participant group—except for the non-binary gender category—is heterogeneous in terms of gender, with only a small percentage difference between men and women. The most notable difference can be seen in users' dominant hand, with left-handed individuals making up only 6.15% of the group's total population.

Thanks to the test, a total of 975 animations were obtained (195 animations each for waving, pointing, sitting, fighting, and running).

4.6.4. Conclusions of the IA Models Comparison

In terms of the AI models, it has been observed that neural network models did not achieve the expected results, likely due to a combination of insufficient data and simplification of the neural networks to ensure shorter training times and faster inference computations.

However, the Random Forest model has yielded the best results, being the fastest model (both in inference and training), the lightest model, and the most adaptable. This model can be used in systems with limited resources, such as standalone VR glasses like Meta Quest or more powerful computers, and even in AI ecosystems due to its possible translation to TensorFlow already implemented.

In the analysis of the Random Forest results, it has been observed that the model places significant importance on the y-coordinate of the left leg and the y-coordinate of the left foot in the second time interval. This may indicate that gestures heavily depend on leg positions to distinguish between running, dancing, waving, or pointing.

Also, there is a slight confusion between the gestures of fighting and waving. This may be due to the rapid hand movements in both types of gestures, which, due to the lack of fingers in the suit, can lead to confusion between a punch and a wave.

4.6.5. Final Application Conclusions

The final application is a demo that connects to a server to display the results.

The negative aspect of this application is the way it connects to the server where the model is hosted, as it makes direct web requests to the Tensor Serving server, making it dependent on that server.

Finally, the final application fulfills its purpose of showing the user the predicted animation with minimal latency.

4.6.6. Limitations

This study has faced a series of limitations, including the scarcity of public datasets, imbalance in types of animations, lack of more data from real users, training issues with neural network models, and data shortages for those same models.

The limitation of dataset scarcity is not only due to the lack of datasets in number but also to the lack of gestures related to non-verbal communication. Most

available datasets focus on sports or very specific themes, with few general-purpose datasets. Another limitation of these datasets has been the ease of adaptation or conversion to a format compatible with the motion capture suit used in the study.

Regarding the imbalance of animation types, the issue arises from the duration of Mixamo's dance animations. Following the standardization process, the various dance animations are multiplied by a significantly larger number than those of other types. This was not resolved with the data collected from real users, as even though a large number of new animations were obtained, after standardization, dance animations still predominated significantly. This can be seen in Figures ?? and ??.

The neural network models did not yield the expected results, which may be due to the lack of data. Although there appears to be a sufficient number of samples at first glance, it is not enough for training neural networks. Additionally, hardware limitations for training more complex models and the time constraints of this study did not allow for longer training sessions or more extensive hyperparameter searches.

All these limitations have led to the following future work plan.

4.7. Future Work

The future work of this study can focus on several efforts:

- Improve the dataset by including people with functional diversity, increasing the number of samples per gesture, considering finger movements, and adding new categories. This could not only enhance the performance of neural network models but also provide greater differentiation for better generalization.
- Investigate other [AI](#) models that could improve classification accuracy and enable classification not only of gestures but also of emotions. This could help [NPCs](#) react more naturally to user gestures, making interactions more immersive.
- Standardize the server to be compatible with other [AI](#) technologies beyond TensorFlow. This would allow for the integration of emerging technologies into existing applications.
- Implement another model that considers not only the current predicted gesture but also the entire context to advance the use of non-verbal communication with [NPCs](#).
- Evaluation of the final result with users.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

AGIS, R. A., GOTTIFREDI, S. y GARCÍA, A. J. An event-driven behavior trees extension to facilitate non-player multi-agent coordination in video games. *Expert Systems with Applications*, vol. 155, página 113457, 2020. ISSN 0957-4174.

AHN, M., BROHAN, A., BROWN, N., CHEBOTAR, Y., CORTES, O., DAVID, B., FINN, C., FU, C., GOPALAKRISHNAN, K., HAUSMAN, K., HERZOG, A., HO, D., HSU, J., IBARZ, J., ICHTER, B., IRPAN, A., JANG, E., RUANO, R. J., JEFFREY, K., JESMONTH, S., JOSHI, N. J., JULIAN, R., KALASHNIKOV, D., KUANG, Y., LEE, K.-H., LEVINE, S., LU, Y., LUU, L., PARADA, C., PASTOR, P., QUIAMBAO, J., RAO, K., RETTINGHOUSE, J., REYES, D., SERMANET, P., SIEVERS, N., TAN, C., TOSHEV, A., VANHOUCKE, V., XIA, F., XIAO, T., XU, P., XU, S., YAN, M. y ZENG, A. Do as i can, not as i say: Grounding language in robotic affordances. 2022.

BANGOR, A., KORTUM, P. T. y MILLER, J. T. An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, vol. 24(6), páginas 574–594, 2008.

BEN-ARI, M. y MONDADA, F. *Finite State Machines*, páginas 55–61. Springer International Publishing, Cham, 2018. ISBN 978-3-319-62533-1.

BENGIO, Y., DUCHARME, R., VINCENT, P. y JANVIN, C. A neural probabilistic language model. *J. Mach. Learn. Res.*, vol. 3(null), página 1137–1155, 2003. ISSN 1532-4435.

- BERGER, A. L., DELLA PIETRA, S. A. y DELLA PIETRA, V. J. A maximum entropy approach to natural language processing. *Computational Linguistics*, vol. 22(1), páginas 39–71, 1996.
- BROOKE, J. SUS-A quick and dirty usability scale. *Usability evaluation in industry*, vol. 189(194), 1996.
- CHO, K., VAN MERRIËNBOER, B., GULCEHRE, C., BAHDANAU, D., BOUGARES, F., SCHWENK, H. y BENGIO, Y. Learning phrase representations using RNN encoder–decoder for statistical machine translation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (editado por A. Moschitti, B. Pang y W. Daelemans), páginas 1724–1734. Association for Computational Linguistics, Doha, Qatar, 2014.
- CHOI, J.-W., KIM, H., ONG, H., YOON, Y., JANG, M., KIM, D. y KIM, J. Reac-tree: Hierarchical llm agent trees with control flow for long-horizon task planning. 2026.
- COLLEDANCHISE, M. y OGREN, P. How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees. *IEEE Transactions on Robotics*, vol. 33, páginas 372–389, 2016.
- FATHONI, K., HAKKUN, R. Y. y NURHADI, H. A. T. Finite state machines for building believable non-playable character in the game of khalid ibn al-walid. *Journal of Physics: Conference Series*, vol. 1577(1), página 012018, 2020.
- FAUZI, R., HARIADI, M., NUGROHO, S. y LUBIS, M. Defense behavior of real time strategy games: Comparison between hfsm and fsm. *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 13(2), páginas 634–642, 2019. ISSN 2502-4752. Publisher Copyright: © 2019 Institute of Advanced Engineering and Science. All rights reserved.
- GAJARDO, I., BESOAIN, F. y BARRIGA, N. A. Introduction to behavior algorithms for fighting games. En *2019 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*, página 1–6. IEEE, 2019.
- GANAPATHI, A., SIVAKUMAR, P., ELANGO, A., GUPTA, H. y PANDA, N. Exploring npc behaviors in games through finite automata. En *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, páginas 1–8. 2024.
- GUGLIERMO, S., CÁCERES DOMÍNGUEZ, D., IANNOTTA, M., STOYANOV, T. y SCHAFFERNICHT, E. Evaluating behavior trees. *Robotics and Autonomous Systems*, vol. 178, página 104714, 2024. ISSN 0921-8890.
- GUO, S. y HOU, S. Analyzing Approaches to Extending and Implementing Behavior Trees in Computer Games. *Applied and Computational Engineering*, vol. 199(1), páginas 139–153, 2025.

- HERBERT, O. M., PÉREZ-GRANADOS, D., RUIZ, M. A. O., CADENA MARTÍNEZ, R., GUTIÉRREZ, C. A. G. y ANTUÑANO, M. A. Z. Static and dynamic hand gestures: A review of techniques of virtual reality manipulation. *Sensors*, vol. 24(12), página 3760, 2024.
- HOFFMEISTER, L. M., SCASSELLATI, B. y RAKITA, D. Towards zero-knowledge task planning via a language-based approach. 2026.
- HU, W., ZHANG, Q. y MAO, Y. Component-based hierarchical state machine — a reusable and flexible game ai technology. 2011.
- HYZY, M., BOND, R., MULVENNA, M., BAI, L., DIX, A., LEIGH, S. y HUNT, S. System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis. *JMIR mHealth and uHealth*, vol. 10(8), página e37290, 2022.
- IOVINO, M., FÖRSTER, J., FALCO, P., CHUNG, J. J., SIEGWART, R. y SMITH, C. Comparison between behavior trees and finite state machines. *IEEE Transactions on Automation Science and Engineering*, vol. 22, páginas 21098–21117, 2025.
- IOVINO, M., SCUKINS, E., STYRUD, J., ÖGREN, P. y SMITH, C. A survey of behavior trees in robotics and ai. 2020.
- IOVINO, M., SCUKINS, E., STYRUD, J., ÖGREN, P. y SMITH, C. A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems*, vol. 154, página 104096, 2022. ISSN 0921-8890.
- JELINEK, F. y MERCER, R. L. Interpolated estimation of Markov source parameters from sparse data. En *Proc. Workshop on Pattern Recognition in Practice*. 1980.
- KIMDONGJU y KOHSEOK-JOO. Intelligent npc ai based on fsm and reinforcement learning: Implementation study of a 2d mobile idle rpg game. *The Transactions of the Korea Information Processing Society*, vol. 14(6), páginas 480–488, 2025.
- LEE, M. An artificial intelligence evaluation on fsm-based game npc. *Journal of Korea Game Society*, vol. 14, páginas 127–135, 2014.
- LYKOV, A. y TSETSERUKOU, D. Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model. 2023.
- NEVEROVA, N., WOLF, C., PACI, G., SOMMAVILLA, G., TAYLOR, G. W. y NEBOUT, F. A multi-scale approach to gesture detection and recognition. 2013.
- NUR FITRIANI DEWI, E., RACHMAN, A. y ZAHIR, H. Implementation of hierarchical finite state machine for controlling 2d character animation in action video game. páginas 1–6. 2023.
- PARTLAN, N., SOTO, L., HOWE, J., SHRIVASTAVA, S., EL-NASR, M. S. y MARSELLA, S. Evolvingbehavior: Towards co-creative evolution of behavior trees for game npcs. 2022.

- PERES, S. C., PHAM, T. y PHILLIPS, R. Validation of the system usability scale (sus): Sus in the wild. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 57(1), páginas 192–196, 2013.
- SEKHAVAT, Y. Behavior trees for computer games. *International Journal on Artificial Intelligence Tools*, vol. 26, página 1730001, 2017.
- SONG, C. H., WU, J., WASHINGTON, C., SADLER, B. M., CHAO, W.-L. y SU, Y. Llm-planner: Few-shot grounded planning for embodied agents with large language models. 2023.
- THOMPSON, T. Cyber demons. <https://www.gamedeveloper.com/design/cyber-demons-the-ai-of-doom-2016->, 2018. Accessed: 2026-8-27.
- TREMBLAY, H., PONTON, D., GONZALEZ, L., FRANCILLETTE, Y. y BOUCHARD, B. Breaking down the challenge: A comprehensive framework for evaluating enemy difficulty with automated behavior tree analytics. *Entertainment Computing*, vol. 57, página 101096, 2026. ISSN 1875-9521.
- VALMEEKAM, K., MARQUEZ, M., OLMO, A., SREEDHARAN, S. y KAMBHAMPATI, S. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. 2023.
- YANNAKAKIS, M. *Hierarchical State Machines*, vol. 1872, páginas 315–330. 2001. ISBN 978-3-540-67823-6.
- ÖGREN, P. y SPRAGUE, C. I. Behavior trees in robot control systems. *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5(1), página 81–107, 2022. ISSN 2573-5144.

JSON Schema para los Behavior Trees generados

Listing A.1: JSON Schema (Draft 2020-12) de los Behavior Trees generados

```

1  {
2    "$schema": "https://json-schema.org/draft/2020-12/schema",
3    "$id": "https://example.com/behaviour-tree.schema.json",
4    "title": "Behaviour Tree",
5    "type": "object",
6    "required": ["Blackboard", "Node"],
7    "additionalProperties": false,
8
9    "properties": {
10     "Blackboard": {
11       "type": "array",
12       "description": "Blackboard",
13       "items": {
14         "type": "object",
15         "additionalProperties": {
16           "type": "string",
17           "enum": ["Bool", "Int", "Float", "Vector",
18                  ↪ "Object", "String"]
19         }
20       }
21     },
22     "Node": {
23       "$ref": "#/$defs/TreeNodeWrapper"
24     }
25   },
26
27   "$defs": {
28     "TreeNodeWrapper": {
29       "type": "object",
30       "description": "Nodo del arbol",

```

```
31     "minProperties": 1,
32     "maxProperties": 1,
33     "additionalProperties": {
34         "$ref": "#/$defs/TreeNode"
35     }
36 },
37
38 "TreeNode": {
39     "type": "object",
40     "required": ["Type", "Decorators", "Nodes"],
41     "additionalProperties": false,
42
43     "properties": {
44         "Type": {
45             "type": "string",
46             "enum": ["Selector", "Sequence", "Task", "Parallel"]
47         },
48
49         "Name": {
50             "type": "string",
51             "description": "Nombre opcional del nodo"
52         },
53
54         "Decorators": {
55             "type": "array",
56             "items": {
57                 "type": "object",
58                 "additionalProperties": {
59                     "type": "string"
60                 }
61             }
62         },
63
64         "Task": {
65             "type": "string",
66             "description": "Nombre de la tarea"
67         },
68
69         "BlackboardEntries": {
70             "type": "array",
71             "description": "Entradas del Blackboard usadas por  
↔ el nodo",
72             "items": {
73                 "type": "string"
74             }
75         },
76
77         "Nodes": {
78             "type": "array",
```

```
79         "description": "Nodos hijos",
80         "items": {
81             "$ref": "#/$defs/TreeNodeWrapper"
82         }
83     }
84 }
85 ,
86 "Nodes": {
87     "type": "array",
88     "description": "Nodos hijos",
89     "minItems": 0,
90     "items": {
91         "type" : "object",
92         "required": ["Node"],
93         "additionalProperties" : false,
94         "properties": {
95             "Node": {
96                 "$ref": "#/$defs/TreeNode"
97             }
98         }
99     }
100 },
101
102 "allOf": [
103     {
104         "if": {
105             "properties": { "Type": { "const": "Task" } }
106         },
107         "then": {
108             "required": ["Task"]
109         }
110     }
111 ]
112 }
113 }
114 }
```


Apéndice **B**

Prompt del sistema para el [LLM](#)
generador del pseudocódigo

You are an expert in behavior trees and algorithm design.

Your task is to convert a high-level task description into a structured pseudocode

Input:

- * A task description

Requirements:

- * Use ONLY the provided actions and ONLY the provided conditions. Do NOT invent
- * Produce a deterministic and unambiguous pseudocode.
- * The structure must be easy to parse programmatically.
- * Explicitly represent:
 - Sequence (ordered execution)
 - Selector (fallback / OR logic / conditionals)
 - Conditions (checks)
 - Actions (leaf nodes)
- * Avoid natural language ambiguity. Use a strict format.

Pseudocode Format Rules:

- * Use uppercase keywords: SEQUENCE, SELECTOR, IF, THEN, ELSE, END
- * You don't have to use all the keywords, only use the necessary ones.
- * Indentation must reflect hierarchy
- * Each condition must be written as: IF condition_name. Use ONLY the names from
- * Each action must be written as: DO action_name. Use ONLY the names from the "
- * ONLY blocks of SEQUENCE and SELECTOR must always be explicitly closed with EN
- * No free text explanations, only pseudocode

Behavior Tree Mapping:

- * SEQUENCE: executes children in order until one fails. Perfect for when action
- * SELECTOR: executes children until one succeeds. Perfect for when you have to
- * IF condition THEN ... ELSE ... : conditional branching

Reasoning Procedure:

Before generating the pseudocode, internally perform the following reasoning.

Do NOT output these steps.

Phase 1

Understand the global objective.

Identify:

- * the final goal;
- * mandatory subtasks;
- * optional subtasks.

Phase 2

Analyse the available actions.

For every action determine internally:

- * what it achieves;
- * when it is useful;

JSON de dependencias de tareas en un Behavior Tree

Listing C.1: Ejemplo del formato que deben seguir los JSON con el orden de dependencias de tareas

```

1 {
2   {
3     "Tasks" : [
4       {
5         "Name" : "HasLineOfSight?",
6         "Childs" : [
7           {
8             "Name" : "ChasePlayer",
9             "Childs" : [
10              {
11                "Name" : "MoveTo",
12                "Childs" : [
13                  {
14                    "Name" : "IsItNear?",
15                    "Childs" : [
16                      {
17                        "Name" : "Attack",
18                        "Childs" : []
19                      }
20                    ]
21                  }
22                ]
23              }
24            ]
25          }
26        ]
27      },
28      {
29        "Name" : "SelectWaypoint",
30        "Childs" : [

```

```
31      {
32          "Name" : "MoveTo",
33          "Childs" : []
34      }
35  ]
36 }
37 ]
38 }
39 }
```


Apéndice **D**

Formulario de opiniones de la herramienta

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

Herramienta para la generación de Behavior Trees

* Indica que la pregunta es obligatoria

1. Usuario *

Usabilidad de la herramienta

En esta sección se le realizarán preguntas sobre la usabilidad de la herramienta.

2. Creo que me gustaría utilizar este sistema con frecuencia *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

3. Encontré el sistema innecesariamente complejo *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

4. Pensé que el sistema era fácil de usar *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

5. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

6. Encontré que las funciones de este sistema estaban bien integradas *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

Figura D.1: Formulario de recogida de datos demográficos (primera parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

7. Encontré que las funciones de este sistema estaban bien integradas *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

8. Pensé que había demasiada inconsistencia en este sistema *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

9. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

10. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

11. Encontré el sistema muy complicado de usar *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

12. Me sentí muy seguro usando el sistema *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

Figura D.2: Formulario de recogida de datos demográficos (segunda parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

13. Necesitaba aprender muchas cosas antes de empezar con este sistema *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

Calidad de la herramienta

14. El tiempo de generación de BTs ha sido adecuado *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

15. La ejecución de los BTs generados ha resultado ser la esperada *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

16. La calidad de los BTs generados es inferior a los generados por un humano *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

17. ¿Los BTs generados han introducido más nodos de los necesarios? Contando todos los BTs generados, ¿cuántos?

18. ¿Cómo de distintos son los BTs creados por ti con los BTs generados?

Figura D.3: Formulario de recogida de datos demográficos (tercera parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

19. Los BTs generados son un buen punto de partida a la hora de crear BTs *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

20. ¿Consideras necesaria la intervención humana en esta herramienta para poder refinar los resultados o los BTs generados han sido lo suficientemente buenos como para no tener que modificarlos?

21. Los BTs generados se pueden modificar fácilmente *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

22. Esta herramienta podría hacer que disminuya la carga de trabajo de un diseñador *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

Opiniones

En esta sección es libre de expresar cualquier opinión respecto a los BTs generados o la herramienta.

23. Respuesta libre

Este contenido no ha sido creado ni aprobado por Google.

Google Formularios

Figura D.4: Formulario de recogida de datos demográficos (cuarta parte)

Glosario

Abstract Factory	El patrón Abstract Factory es un patrón de programación que consiste en mantener una clase que permita crear instancias de objetos relacionados gracias al polimorfismo.
AI	Artificial Intelligence
Almacén vectorial	Un almacén vectorial (o vector store) es una base de datos especializada en el almacenaje de la información en forma de vectores (o embeddings), la cual permite recuperar los datos por similitud matemática a otros vectores.
API	Application Programming Interface
Behavior Tree	Un Behavior Tree es un modelo de ejecución de tareas de forma jerárquica basado en árboles usada en el campo de la robótica y la inteligencia artificial
Blackboard	La Blackboard es la memoria compartida usada en un Behavior Tree. Ésta funciona como un diccionario de pares clave-valor, que pueden ser variables de entrada/salida.
CSV	Comma Separated Values
Embedding	Un embedding es un medio para representar la información compleja de forma numérica. Esto se consigue trasladando la información a un espacio vectorial en el que a cada unidad se le determina unas coordenadas según el contexto que tiene a su alrededor. Así, conceptos cercanos se mantienen matemáticamente cerca y son más sencillos de relacionar.

FAISS	FAISS (Facebook AI Similarity Search) es una librería de Meta que permite la búsqueda y el agrupamiento eficiente de vectores densos con el fin de facilitar la búsqueda por similitud.
Fine-tuning	El Fine-tuning es una técnica de Machine Learning que consiste en reentrenar ciertas partes de modelos ya preentrenados con nuevos datos con el objetivo de especializarlo en un tema concreto.
IA	Inteligencia Artificial
LLM	Un LLM (Large Language Model) es un modelo de aprendizaje automático profundo (deep learning) que ha sido entrenado con grandes cantidades de texto con propósito general de predicción de palabras.
metaverse	The metaverse is a set of digital spaces to socialize, learn, play and do other activities. (Definition by the company Meta)
NPC	Non Playable Character
Ollama	Ollama es una aplicación basada en el motor de inferencia "llama.cpp" que permite descargar y ejecutar modelos localmente.
RAG	RAG (Retrieval Augmented Generation) es una técnica de Inteligencia Artificial que permite mejorar las respuestas de modelos de lenguaje buscando información en documentos externos que tengan que ver con la pregunta del usuario.
Random Forest	Un random forest es un algoritmo de aprendizaje automático que utiliza múltiples árboles de decisión para mejorar la precisión y la robustez de las predicciones
RPG	Rol-Playing Game
Singleton	El patrón Singleton es un patrón de programación que consiste en garantizar que una clase tenga una instancia única y que se pueda acceder globalmente a ésta.
VR	Realidad Virtual

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)